CE396

การออกแบบระบบฐานข้อมูล

สัปดาห์ที่ 8 — การสืบค้นข้อมูลพื้นฐาน

INSERT · SELECT · WHERE · LIKE · ORDER BY · GROUP BY

คาบบรรยาย 2 ชั่วโมง · คาบปฏิบัติการ 2 ชั่วโมง (ลงมือทำใน pgAdmin เป็นหลัก)

ภาคการศึกษาที่ 1 ปีการศึกษา 2569

อาจารย์สนายุ จินตนาวรรณกุล

วันนี้เราจะเรียนอะไร

เรียน **6 คำสั่ง** ผ่าน **68 ตัวอย่าง** บนฐานข้อมูลที่สร้างใหม่ทั้งหมดในวันนี้

คำสั่ง	ตอบคำถามว่า	จำนวนตัวอย่าง
`INSERT`	จะเอาข้อมูลเข้าฐานข้อมูลได้อย่างไร	9
`SELECT`	จะดึงข้อมูลออกมาดูได้อย่างไร	9
`WHERE`	จะเอาเฉพาะแถวที่ต้องการได้อย่างไร	15
`LIKE`	จะค้นหาข้อความแบบ "คล้าย ๆ" ได้อย่างไร	13
`ORDER BY`	จะเรียงลำดับผลลัพธ์ได้อย่างไร	11
`GROUP BY`	จะสรุปยอดเป็นกลุ่ม ๆ ได้อย่างไร	15

วันนี้เน้น **ทำซ้ำเยอะ ๆ จนคล่อง** ไม่ได้เน้นเนื้อหากว้าง
① ผู้สอนรันสดให้ดู 31 ตัวอย่าง ② ที่เหลืออยู่ในไฟล์ `.sql` ให้รันเองตามจังหวะ
③ ทุกตัวอย่างมีเขียนไว้ว่า "ต้องได้กี่แถว" ตรวจคำตอบตัวเองได้ทันที

ผลลัพธ์การเรียนรู้ของสัปดาห์นี้

ระดับการเรียนรู้	เมื่อจบคาบนักศึกษาจะ...	วัดผลจาก
จำ	บอกหน้าที่ของ 6 คำสั่ง และลำดับที่ PostgreSQL ประมวลผลได้	ชวนคิดในคาบ
เข้าใจ	อธิบายได้ว่าทำไม `WHERE email = NULL` จึงได้ 0 แถวโดยไม่มี error	ชวนคิดข้อที่ 2
ประยุกต์	เขียนคำสั่งสืบค้นตอบโจทย์ธุรกิจที่กำหนดให้ได้	แบบฝึกหัด 15 ข้อ
วิเคราะห์	อ่านข้อความ error แล้วระบุได้ว่าละเมิดข้อกำหนดตัวใด	ชวนคิดข้อที่ 1
ประเมินค่า	เลือกได้ว่าเงื่อนไขหนึ่ง ๆ ควรอยู่ใน `WHERE` หรือ `HAVING`	ชวนคิดข้อที่ 4
สร้างสรรค์	ประกอบคำสั่งเดียวที่ใช้ครบทุก clause ตอบคำถามผู้บริหาร	ชวนคิดข้อที่ 5

สอดคล้องกับ CLO4 — เขียนคำสั่ง SQL เพื่อสร้าง จัดการ และสืบค้นข้อมูลจากฐานข้อมูลเชิงสัมพันธ์ได้

เริ่มจากศูนย์ — สัปดาห์นี้ไม่ต่อจากใคร

ข่าวดีสำหรับทุกคน

- วันนี้เรา **สร้างฐานข้อมูลใหม่ทั้งหมด** ชื่อ `coffeedb`
- ไม่ต้องใช้ `shopdb` หรือฐานข้อมูลใด ๆ ของสัปดาห์ที่ 1–7 เลย
- ใครเครื่องพัง ใครลบฐานข้อมูลทิ้งไปแล้ว ใครขาดเรียนมาหลายสัปดาห์ — **วันนี้ทำได้ครบ 100%**

สิ่งที่ต้องมีติดเครื่องแค่ 2 อย่าง

1. **PostgreSQL 16 ขึ้นไป** (ในคาบนี้ใช้เวอร์ชัน 17)
2. **pgAdmin 4**

ขั้นตอนเริ่มต้น

เปิดไฟล์ `โค้ด_สัปดาห์ที่8_CE396/01-setup-coffeedb.sql` → สร้างฐานข้อมูล `coffeedb` → วางสคริปต์ → กดรัน

> ⚠️ **ใช้ pgAdmin เป็นหลัก** ไม่แนะนำ `psql` บน Windows
> เพราะหน้าต่าง console มักตั้งตัวเข้ารหัสไม่ตรง ทำให้ `ลาเต้` กลายเป็นตัวประหลาด
> ถ้าจำเป็นต้องใช้จริง ๆ ให้พิมพ์ `chcp 65001` ก่อน แล้วสั่ง `\encoding UTF8`

ส่วนที่ 1

ร้านกาแฟ "บ้านละมุน" และตาราง 3 ใบ

กรณีศึกษา: ร้านกาแฟบ้านละมุน

menu
รายการสินค้า 20 รายการ
5 หมวด · ราคา 55–110 บาท

customer
สมาชิกสะสมแต้ม 12 คน
3 ระดับ · silver / gold / platinum

sale_item
รายการขาย 60 บรรทัด
36 ใบเสร็จ · 7 วัน · 3 สาขา

3 สาขา : สาขาสีลม · สาขารังสิต · สาขาท่าพระจันทร์
บาริสต้า 4 คน · ช่องทางขาย 3 แบบ · วิธีชำระเงิน 3 แบบ · ยอดขายรวม 6,350 บาท

ข้อมูลตั้งต้นทั้งหมด **92 แถว** — เล็กพอให้นับผลลัพธ์ด้วยตาตรวจคำตอบเองได้
แต่ใหญ่พอให้ `GROUP BY` ออกมาเป็นรายงานที่มีความหมายจริง

ตาราง `menu` — พระเอกของ SELECT / WHERE / LIKE / ORDER BY

```sql
CREATE TABLE menu (
    menu_id      integer      GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    menu_code    varchar(8)   NOT NULL,           -- รหัสเมนู เช่น HOT-001
    menu_name_th varchar(60)  NOT NULL,           -- ชื่อไทย
    menu_name_en varchar(60)  NOT NULL,           -- ชื่ออังกฤษ
    category     varchar(20)  NOT NULL,           -- หมวด 5 หมวด
    price        numeric(6,2) NOT NULL,
    cost         numeric(6,2) NOT NULL,
    kcal         smallint,                        -- ★ เป็น NULL ได้ (2 แถว)
    is_signature boolean      NOT NULL DEFAULT false,
    launch_date  date         NOT NULL,
    note         varchar(120),                    -- ★ เป็น NULL ได้ (13 แถว)
    CONSTRAINT uq_menu_code  UNIQUE (menu_code),
    CONSTRAINT ck_menu_price CHECK (price > 0),
    CONSTRAINT ck_menu_cost  CHECK (cost >= 0 AND cost < price),
    CONSTRAINT ck_menu_cat   CHECK (category IN
                ('กาแฟร้อน','กาแฟเย็น','ชา','ขนม','เครื่องดื่มอื่น'))
);
```

คอลัมน์	จะถูกใช้สอนเรื่อง
`menu_name_th`	`LIKE` ภาษาไทย · การเรียงลำดับภาษาไทย
`menu_name_en`	`LIKE` แยกตัวพิมพ์ เทียบกับ `ILIKE`
`category`	`GROUP BY` (5 กลุ่ม) · `IN` / `NOT IN` · error จาก `CHECK`
`price` / `cost`	คอลัมน์คำนวณ · การเรียงที่มีค่าซ้ำ · `SUM` / `AVG` / `MIN` / `MAX`
`kcal` / `note`	ทุกเรื่องที่เกี่ยวกับ `NULL`

ตาราง `customer` — พระเอกของบทเรียนเรื่อง NULL

```sql
CREATE TABLE customer (
    member_code varchar(8)   PRIMARY KEY,         -- รหัสที่คนตั้งเอง ไม่ใช่เลขอัตโนมัติ
    full_name   varchar(80)  NOT NULL,
    nickname    varchar(30),
    phone       varchar(15)  NOT NULL,
    email       varchar(80),                      -- ★ เป็น NULL 3 จาก 12 แถว
    tier        varchar(10)  NOT NULL DEFAULT 'silver',
    points      integer      NOT NULL DEFAULT 0,
    birth_month smallint,
    joined_on   date         NOT NULL DEFAULT CURRENT_DATE,
    promo_note  varchar(60),
    CONSTRAINT uq_cust_phone  UNIQUE (phone),
    CONSTRAINT ck_cust_tier   CHECK (tier IN ('silver','gold','platinum')),
    CONSTRAINT ck_cust_points CHECK (points >= 0),
    CONSTRAINT ck_cust_month  CHECK (birth_month IS NULL OR birth_month BETWEEN 1 AND 12),
    CONSTRAINT ck_cust_phone  CHECK (phone LIKE '0__-___-____')   -- ★ ดูตรงนี้ให้ดี
);
```

สังเกต 2 จุด

- **`CHECK (phone LIKE '0__-___-____')`** — นี่คือ `LIKE` ที่เราจะเรียนกันตอนบ่าย เอามาใช้ใน `CHECK` ได้ด้วย อ่านว่า "เลข 0 · อักขระอะไรก็ได้ 2 ตัว · ขีด · 3 ตัว · ขีด · 4 ตัว"
- **`member_code` เป็นรหัสที่คนตั้งเอง** ไม่ใช่ `IDENTITY` จึงเป็นตารางเดียวที่เราจะสาธิต `INSERT` แบบไม่ระบุชื่อคอลัมน์ได้อย่างปลอดภัย

ตาราง `sale_item` — พระเอกของ GROUP BY

```
CREATE TABLE sale_item (
    sale_item_id integer      GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    receipt_no   varchar(12)  NOT NULL,     sold_at    timestamp    NOT NULL,
    branch       varchar(24)  NOT NULL,     -- 3 กลุ่ม
    channel      varchar(14)  NOT NULL,     -- 3 กลุ่ม
    pay_method   varchar(14)  NOT NULL,     -- 3 กลุ่ม
    barista      varchar(30)  NOT NULL,     -- 4 กลุ่ม

    menu_code    varchar(8)   NOT NULL,     -- ◆ คัดลอกมาจาก menu
    menu_name_th varchar(60)  NOT NULL,     -- ◆ คัดลอกมาจาก menu
    category     varchar(20)  NOT NULL,     -- ◆ คัดลอกมาจาก menu

    qty          smallint     NOT NULL,
    unit_price   numeric(6,2) NOT NULL,     -- ราคา ณ วันที่ขาย
    discount     numeric(6,2) NOT NULL DEFAULT 0,
    member_code  varchar(8),                -- ★ NULL = ลูกค้าทั่วไป (29 จาก 60 แถว)
    remark       varchar(100),              -- ★ NULL 49 จาก 60 แถว
    CONSTRAINT fk_sale_member FOREIGN KEY (member_code) REFERENCES customer(member_code),
    CONSTRAINT uq_sale_line   UNIQUE (receipt_no, menu_code),
    CONSTRAINT ck_sale_qty    CHECK (qty BETWEEN 1 AND 20)
    -- ... และ CHECK อีก 4 ตัว
);
```

ไม่มีคอลัมน์ `line_total` — ยอดของแต่ละบรรทัดต้องเขียน `qty * unit_price - discount` เองทุกครั้ง
จงใจ เพราะจะได้ฝึก 3 เรื่องพร้อมกัน: ① คอลัมน์คำนวณใน `SELECT` ② `ORDER BY` ตามนิพจน์ ③ `SUM()` ของนิพจน์

⚠️ ทำไม `sale_item` ถึงเก็บข้อมูลซ้ำ

เราเพิ่งเรียนเรื่องบรรทัดฐานไปสัปดาห์ที่ 5 — แล้วทำไมตารางนี้ถึงมีข้อมูลซ้ำ?

ไม่ได้ลืม และไม่ได้ออกแบบพลาด มีเหตุผล 3 ข้อ

① `unit_price` ไม่ใช่ข้อมูลซ้ำ

เป็น **ราคา ณ วันที่ขาย** ถ้าพรุ่งนี้ร้านขึ้นราคาลาเต้จาก 70 เป็น 75 บาท
ใบเสร็จของเมื่อวานต้องยังเป็น 70 บาทเหมือนเดิม — การเก็บราคาไว้ในใบเสร็จจึง**ถูกต้อง**

② `menu_name_th` กับ `category` ซ้ำจริง แต่เรายุบไว้ชั่วคราว

เพราะคำสั่ง `JOIN` ยังไม่อยู่ในขอบเขตของสัปดาห์นี้
ตารางหน้าตาแบบนี้มีชื่อเรียกว่า **"ตารางข้อเท็จจริง" (fact table)** — ระบบรายงานยอดขายของจริงใช้กันแบบนี้ทั้งนั้น

③ `member_code` เป็นคีย์นอกจริง แต่เราจะไม่ `JOIN` มันเลยทั้งสัปดาห์

มีไว้เพื่อ ① เป็นคอลัมน์ที่เป็น `NULL` ได้ ② สาธิต error 23503 ③ เป็นสะพานไปสัปดาห์หน้า

> **สัปดาห์หน้าเราจะรู้จัก `JOIN` แล้วกลับมาแก้ตารางนี้ให้เรียบร้อย**
> การรู้ว่า "ตรงไหนซ้ำ และซ้ำเพราะอะไร" สำคัญกว่าการไม่มีอะไรซ้ำเลย

ข้อมูลตั้งต้น 92 แถว — หน้าตาเป็นอย่างไร

`menu` (แสดง 7 จาก 20 แถว)

menu_code	menu_name_th	menu_name_en	category	price	cost	kcal	note
HOT-001	เอสเพรสโซ่	Espresso	กาแฟร้อน	55.00	18.00	10	*(null)*
HOT-002	อเมริกาโน่ร้อน	Hot Americano	กาแฟร้อน	60.00	20.00	15	ลดน้ำตาล 50% ได้
HOT-003	ลาเต้ร้อน	Hot Latte	กาแฟร้อน	70.00	26.00	150	*(null)*
ICE-002	ลาเต้เย็น	Iced Latte	กาแฟเย็น	75.00	27.00	180	*(null)*
ICE-005	ลาเต้มะพร้าวเย็น	Iced Coconut Latte	กาแฟเย็น	95.00	40.00	*(null)*	สูตรใหม่ ยังไม่ได้วัดแคลอรี
TEA-002	ชาเขียวมัทฉะลาเต้	Matcha Green Tea Latte	ชา	90.00	36.00	200	*(null)*
BAK-003	ชีสเค้กมะพร้าว	Coconut Cheesecake	ขนม	110.00	45.00	*(null)*	*(null)*

`sale_item` (แสดง 4 จาก 60 แถว)

receipt_no	sold_at	branch	pay_method	menu_name_th	category	qty	unit_price	discount	member_code
R69-0001	2026-09-14 07:42	สาขาสีลม	เงินสด	ลาเต้ร้อน	กาแฟร้อน	1	70.00	0.00	M-001
R69-0001	2026-09-14 07:42	สาขาสีลม	เงินสด	ครัวซองต์เนยสด	ขนม	2	75.00	0.00	M-001
R69-0002	2026-09-14 08:15	สาขาสีลม	พร้อมเพย์	คาราเมลมัคคิอาโต้	กาแฟเย็น	1	95.00	5.00	*(null)*
R69-0029	2026-09-19 11:55	สาขาท่าพระจันทร์	พร้อมเพย์	คาราเมลมัคคิอาโต้	กาแฟเย็น	3	95.00	15.00	M-006

> `SELECT COUNT(*)` ทั้งสามตารางต้องได้ **20 · 12 · 60** เสมอ ถ้าไม่ตรง ให้รันไฟล์ `02` ส่วนที่ 0 แล้วรันส่วนที่ 1 ใหม่

ส่วนที่ 2

`INSERT` — นำข้อมูลเข้าฐานข้อมูล

ไฟล์ `02-insert.sql`

โครงคำสั่ง `INSERT` 3 แบบ

แบบ	รูปแบบ	ใช้เมื่อไร
ระบุคอลัมน์	`INSERT INTO ตาราง (คอลัมน์...) VALUES (ค่า...);`	**ใช้แบบนี้เสมอ** อ่านรู้เรื่อง ไม่พลาดลำดับ
หลายแถว	`INSERT INTO ตาราง (คอลัมน์...) VALUES (...), (...), (...);`	ใส่ทีละหลายแถว คุยกับเซิร์ฟเวอร์รอบเดียว
ไม่ระบุคอลัมน์	`INSERT INTO ตาราง VALUES (ค่า...);`	⚠️ ต้องเรียงค่าให้ตรงทุกคอลัมน์ · ห้ามใช้กับตารางที่มี `IDENTITY`

```
INSERT INTO menu (menu_code, menu_name_th, menu_name_en, category,
                  price, cost, launch_date)
VALUES ('HOT-007', 'ลาเต้ส้มยูซุร้อน', 'Hot Yuzu Latte', 'กาแฟร้อน',
        95.00, 38.00, DATE '2026-09-22');
```

```
INSERT 0 1
```

ข้อความตอบกลับ `INSERT 0 1` อ่านว่า "เพิ่มสำเร็จ 1 แถว"
เลข 0 ตัวหน้าเป็นของเก่าที่ PostgreSQL ยังคงไว้เพื่อความเข้ากันได้ ไม่ต้องสนใจ
อ่านข้อความตอบกลับทุกครั้ง — นี่คือหลักฐานว่าคำสั่งทำงานจริง

I1–I2 — แถวเดียว และหลายแถวในคำสั่งเดียว

I1 — แถวเดียว ระบุคอลัมน์

```
INSERT INTO menu (menu_code, menu_name_th, menu_name_en, category,
                  price, cost, launch_date)
VALUES ('HOT-007', 'ลาเต้ส้มยูซุร้อน', 'Hot Yuzu Latte', 'กาแฟร้อน',
        95.00, 38.00, DATE '2026-09-22');
```

คอลัมน์ที่เราไม่ได้เขียนถึง (`kcal` , `note`) จะเป็น `NULL`

ส่วน `is_signature` จะเป็น `false` เพราะประกาศ `DEFAULT false` ไว้ตอน `CREATE TABLE`

I2 — หลายแถว คั่นด้วยจุลภาค

```
INSERT INTO menu (menu_code, menu_name_th, menu_name_en, category,
                  price, cost, launch_date) VALUES
('ICE-006', 'ลาเต้ส้มยูซุเย็น', 'Iced Yuzu Latte',  'กาแฟเย็น', 100.00, 40.00, DATE '2026-09-22'),
('TEA-005', 'ชาอู่หลงนมสด',    'Oolong Milk Tea',  'ชา',        80.00, 30.00, DATE '2026-09-22'),
('BAK-004', 'คุกกี้ช็อกชิป',    'Choc Chip Cookie', 'ขนม',       45.00, 16.00, DATE '2026-09-22');
```

```
INSERT 0 3
```

> ทำไมไม่เขียน `INSERT` สามครั้ง? เพราะแบบนี้เดินทางไปเซิร์ฟเวอร์รอบเดียว
> และถ้ามีแถวใดผิดกฎ **ทั้งคำสั่งจะถูกยกเลิกพร้อมกัน** ไม่มีแถวไหนเข้าไปครึ่ง ๆ กลาง ๆ

I3–I4 — ไม่ระบุคอลัมน์ และค่า `DEFAULT`

I3 — ไม่ระบุคอลัมน์ ต้องเรียงค่าให้ตรงทุกตัว

```
INSERT INTO customer VALUES
('M-013', 'ศิริพร ทองดี', 'พร', '081-777-2020', 'siriporn@gmail.com',
 'gold', 600, 4, DATE '2026-09-25', NULL);
```

ลำดับต้องเป็น `member_code, full_name, nickname, phone, email, tier, points, birth_month, joined_on, promo_note`

> ⚠️ **ห้ามใช้กับตาราง `menu` และ `sale_item`** เพราะค่าแรกจะต้องเป็นคอลัมน์ `IDENTITY` ที่ระบบควรสร้างเลขให้เอง · และถ้ามีคนเพิ่มคอลัมน์ในตารางทีหลัง คำสั่งแบบนี้จะพังทันที

I4 — ละคอลัมน์ไว้ = ได้ `DEFAULT`

```
INSERT INTO customer (member_code, full_name, nickname, phone)
VALUES ('M-014', 'ภาณุวัฒน์ ก้องไกล', 'นุ', '081-777-3030');

SELECT member_code, tier, points, joined_on, email
FROM   customer WHERE member_code = 'M-014';
```

```
 member_code |  tier  | points | joined_on  | email
-------------+--------+--------+------------+-------
 M-014       | silver |      0 | 2026-09-28 | [null]
```

> คอลัมน์ที่มี `DEFAULT` ได้ค่าตามที่ประกาศไว้ · คอลัมน์ที่ไม่มี `DEFAULT` และไม่ใช่ `NOT NULL` ได้ `NULL`

I5 — คำสั่ง `DEFAULT` และ `RETURNING`

```
INSERT INTO menu (menu_code, menu_name_th, menu_name_en, category,
                  price, cost, is_signature, launch_date)
VALUES ('OTH-003', 'น้ำมะนาวโซดา', 'Lemon Soda', 'เครื่องดื่มอื่น',
        75.00, 25.00, DEFAULT, DATE '2026-09-26')
RETURNING menu_id, menu_code, menu_name_th, is_signature;
```

```
 menu_id | menu_code | menu_name_th   | is_signature
---------+-----------+----------------+--------------
      25 | OTH-003   | น้ำมะนาวโซดา    | f
```

สองอย่างที่ได้จากตัวอย่างนี้

คำสั่ง	ทำอะไร
`DEFAULT` ในรายการค่า	บอกชัด ๆ ว่า "คอลัมน์นี้ขอใช้ค่าตั้งต้น" — เขียนไว้ให้คนอ่านเห็นเจตนา
`RETURNING`	คืนค่าของแถวที่เพิ่งเพิ่มกลับมาให้ดู

> **`RETURNING` มีประโยชน์มากในงานจริง** เพราะ `menu_id` ถูกสร้างโดยเซิร์ฟเวอร์ โปรแกรมของเราไม่มีทางรู้ล่วงหน้าว่าจะได้เลขอะไร ถ้าต้องเอาเลขนั้นไปใช้ต่อทันที `RETURNING` คือวิธีที่สั้นและปลอดภัยที่สุด

I6 — `NULL` ไม่เท่ากับ `''` และไม่เท่ากับ `0`

```
INSERT INTO menu (menu_code, menu_name_th, menu_name_en, category,
                  price, cost, kcal, launch_date, note)
VALUES ('BAK-005', 'พายสับปะรด', 'Pineapple Pie', 'ขนม',
        70.00, 28.00, NULL, DATE '2026-09-26', '');
```

```
SELECT kcal, note, kcal IS NULL AS แคลอรีเป็นNULL,
       note IS NULL AS หมายเหตุเป็นNULL, note = '' AS หมายเหตุว่าง,
       length(note) AS ความยาว
FROM menu WHERE menu_code = 'BAK-005';
```

```
  kcal   | note | แคลอรีเป็นNULL | หมายเหตุเป็นNULL | หมายเหตุว่าง | ความยาว
---------+------+----------------+------------------+--------------+---------
 [null] |      | t              | f                | t            |       0
```

ค่า	ความหมาย	ตัวอย่างในชีวิตจริง
`NULL`	**ไม่รู้ / ยังไม่มีข้อมูล**	ยังไม่ได้วัดแคลอรีของเมนูนี้
`''`	รู้แน่ ๆ ว่าเป็นข้อความว่าง	ถามแล้ว ลูกค้าไม่มีหมายเหตุจะฝาก
`0`	รู้แน่ ๆ ว่าเป็นศูนย์	วัดแล้วได้ศูนย์แคลอรีจริง

> สามอย่างนี้เก็บคนละความหมาย และ**เปรียบเทียบด้วยวิธีคนละแบบ** — จำให้แม่น เจอแน่ในข้อสอบ

I7–I9 — อ่าน error ให้ออก

เวลาคำสั่งพัง ให้อ่าน **3 อย่าง** เสมอ: ① เลข SQLSTATE ② ชื่อ constraint ③ บรรทัด `DETAIL:`

SQLSTATE	ละเมิดอะไร	ตัวอย่างคำสั่งที่พัง
`23502`	`NOT NULL`	ลืมใส่ `menu_name_th`
`23503`	`FOREIGN KEY`	ใส่ `member_code = 'M-999'` ที่ไม่มีในตาราง `customer`
`23505`	`UNIQUE`	ใส่ `menu_code = 'HOT-003'` ที่มีอยู่แล้ว
`23514`	`CHECK`	ใส่ `price = 0` ทั้งที่กำหนด `CHECK (price > 0)`

```
INSERT INTO menu (menu_code, menu_name_th, menu_name_en, category, price, cost, launch_date)
VALUES ('HOT-003', 'ลาเต้ร้อนสูตรใหม่', 'Hot Latte v2', 'กาแฟร้อน', 75.00, 28.00, CURRENT_DATE);
```

```
ERROR:  duplicate key value violates unique constraint "uq_menu_code"
DETAIL: Key (menu_code)=(HOT-003) already exists.
SQL state: 23505
```

ชื่อ constraint คือเบาะแสที่ดีที่สุด — เห็น `uq_menu_code` ปุ๊บ เปิดไฟล์ `01` ดูก็รู้ทันทีว่าผิดกฎข้อไหน

นี่คือเหตุผลที่เราตั้งชื่อ constraint เองทุกตัว แทนที่จะปล่อยให้ PostgreSQL ตั้งชื่อให้

? ชวนคิดข้อที่ 1 — คำสั่งไหนพัง และพังเพราะอะไร

ทั้ง 4 คำสั่งนี้รันบนฐานข้อมูล `coffeedb` ที่มีข้อมูลตั้งต้นครบแล้ว

ทายว่าอันไหนสำเร็จ อันไหนพัง และถ้าพังคือละเมิดข้อกำหนดตัวไหน

```sql
-- (ก)
INSERT INTO menu (menu_code, menu_name_th, menu_name_en, category, price, cost, launch_date)
VALUES ('HOT-009', 'ลาเต้ชาไทย', 'Thai Tea Latte', 'กาแฟร้อน', 85.00, 90.00, CURRENT_DATE);

-- (ข)
INSERT INTO customer (member_code, full_name, phone)
VALUES ('M-050', 'ทดสอบ ระบบ', '0899998888');

-- (ค)
INSERT INTO menu (menu_code, menu_name_th, menu_name_en, category, price, cost, launch_date)
VALUES ('DRK-001', 'สมูทตี้มะม่วง', 'Mango Smoothie', 'สมูทตี้', 90.00, 35.00, CURRENT_DATE);

-- (ง)
INSERT INTO customer (member_code, full_name, phone, tier)
VALUES ('M-051', 'ทดสอบ สอง', '089-777-6666', 'diamond');
```

ใบ้: ทั้ง 4 ข้อพังหมด — แต่ละข้อด้วยเหตุผลคนละอย่าง

✅ แนวคำตอบข้อที่ 1

ข้อ	ผล	SQLSTATE	เพราะอะไร
ก	❌	`23514`	`ck_menu_cost` กำหนด `cost < price` แต่ใส่ต้นทุน 90 ซึ่งสูงกว่าราคา 85
ข	❌	`23514`	`ck_cust_phone` กำหนด `phone LIKE '0__-___-____'` แต่ `'0899998888'` ไม่มีขีด
ค	❌	`23514`	`ck_menu_cat` อนุญาตแค่ 5 หมวด ไม่มีหมวด `'สมูทตี้'`
ง	❌	`23514`	`ck_cust_tier` อนุญาตแค่ `silver` / `gold` / `platinum` ไม่มี `'diamond'`

บทเรียน 3 ข้อจากคำถามนี้

1. `CHECK` **ทำงานหนักกว่าที่คิด** — ทั้ง 4 ข้อเป็น 23514 เหมือนกันหมด แต่คนละ constraint ถ้าไม่อ่าน**ชื่อ** constraint จะไม่มีทางรู้ว่าผิดข้อไหน
2. **ข้อกำหนดคือการออกแบบ** — เราตัดสินใจตั้งแต่ตอน `CREATE TABLE` ว่าข้อมูลแบบไหนรับไม่ได้ ฐานข้อมูลจึงปฏิเสธให้เราตั้งแต่ประตูแรก ไม่ต้องรอไปพบปัญหาในรายงานทีหลัง
3. **ถ้าอยากเพิ่มหมวด** `'สมูทตี้'` **จริง ๆ** ต้องไปแก้ `CHECK` ในตารางก่อน ไม่ใช่หาทางเลี่ยงด้วยการยัดข้อมูลเข้าไปทางอื่น

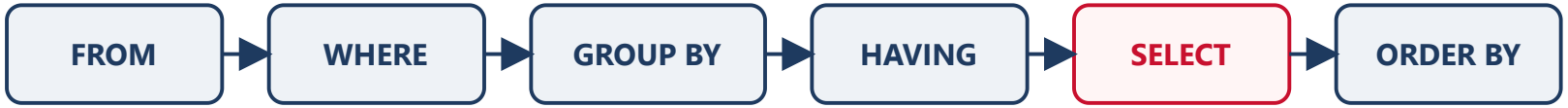
> ลองรันทั้ง 4 ข้อในไฟล์ `02-insert.sql` ส่วนที่ 3 เพื่อเห็นข้อความจริง

ส่วนที่ 3

`SELECT` พื้นฐาน

ไฟล์ `03-select-where.sql`

แผนภาพที่จะอ้างถึงอีก 5 ครั้งวันนี้



▲ ลำดับที่ PostgreSQL ทำงานจริง

แต่ลำดับที่เราเขียนคือ SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY → LIMIT สองลำดับนี้ไม่เหมือนกัน และความต่างนี้อธิบายเรื่องแปลกอีกหลายเรื่องที่เราจะเจอวันนี้

> จำไว้ว่า `SELECT` **ทำงานเกือบท้ายสุด** ทั้งที่เราเขียนไว้บนสุด
> อีก 40 นาทีข้างหน้า เราจะกลับมาที่ภาพนี้เพื่ออธิบายว่า
> ทำไม `ORDER BY` ใช้ชื่อแทนได้ แต่ `WHERE` ใช้ไม่ได้

S1–S3 — เลือกคอลัมน์ และตั้งชื่อด้วย `AS`

```sql
SELECT * FROM menu;                                -- 20 แถว × 11 คอลัมน์
SELECT menu_name_th, category, price FROM menu;    -- 20 แถว × 3 คอลัมน์
SELECT price, menu_name_th FROM menu;              -- สลับลำดับก็ได้
```

⚠️ `*` **เหมาะกับการสำรวจข้อมูลเท่านั้น** เวลาเขียนโปรแกรมจริงให้ระบุชื่อคอลัมน์เสมอ
เพราะถ้ามีคนเพิ่มคอลัมน์ในตารางทีหลัง โปรแกรมของเราจะได้ข้อมูลเกินมาโดยไม่รู้ตัว

```sql
SELECT menu_name_th AS "ชื่อเมนู",
       price        AS "ราคา (บาท)",
       category     AS หมวด
FROM   menu;
```

```
      ชื่อเมนู        | ราคา (บาท) |    หมวด
----------------------+------------+-----------
 เอสเพรสโซ่           |      55.00 | กาแฟร้อน
 อเมริกาโน่ร้อน       |      60.00 | กาแฟร้อน
 ...
```

กรณี	ต้องครอบด้วย `"..."` ไหม
ชื่อคำเดียว ไม่มีเว้นวรรค เช่น `หมวด`	ไม่ต้อง
มีเว้นวรรคหรือวงเล็บ เช่น `ราคา (บาท)`	**ต้องครอบ**

`AS` เปลี่ยนแค่**หัวตารางที่แสดงผล** ไม่ได้เปลี่ยนชื่อคอลัมน์จริงในฐานข้อมูล

S4–S5 — คอลัมน์ที่คำนวณขึ้นมาใหม่

```
SELECT menu_name_th, price, cost,
       price - cost AS กำไรต่อแก้ว
FROM   menu;
```

```
   menu_name_th    | price | cost  | กำไรต่อแก้ว
-------------------+-------+-------+-------------
 เอสเพรสโซ่         | 55.00 | 18.00 |       37.00
 ลาเต้ร้อน          | 70.00 | 26.00 |       44.00
 ชีสเค้กมะพร้าว     |110.00 | 45.00 |       65.00   ← กำไรสูงสุด
```

```
SELECT menu_name_th, price,
       ROUND((price - cost) / price * 100, 1) AS เปอร์เซ็นต์กำไร
FROM   menu;
```

```
   menu_name_th    | price | เปอร์เซ็นต์กำไร
-------------------+-------+-----------------
 เอสเพรสโซ่         | 55.00 |            67.3
 ลาเต้ร้อน          | 70.00 |            62.9
```

คอลัมน์แบบนี้ไม่มีอยู่จริงในตาราง — เกิดขึ้นสด ๆ ตอนคำสั่งทำงาน แล้วหายไปเมื่อคำสั่งจบ

`ROUND(ตัวเลข, จำนวนทศนิยม)` · ที่หารแล้วได้ทศนิยมถูกต้องเพราะ `price` เป็นชนิด `numeric` ถ้าเป็น `integer` ทั้งคู่ PostgreSQL จะหารแบบปัดเศษทิ้ง `44/70` จะได้ `0`

S6 — `||` ต่อข้อความ และกับดัก NULL

```
SELECT menu_name_th || ' (' || menu_name_en || ')' AS ชื่อเต็ม FROM menu;
```

```
          ชื่อเต็ม
---------------------------
 ลาเต้ร้อน (Hot Latte)
 ชาไทยเย็น (Thai Iced Tea)
```

⚠️ **แต่ถ้าในนิพจน์มี `NULL` แม้แต่ตัวเดียว...**

```
SELECT menu_code, note, menu_name_th || ' — ' || note AS รายละเอียด FROM menu;
```

```
 menu_code |        note         |              รายละเอียด
-----------+---------------------+-----------------------------------
 HOT-001   | [null]              | [null]              ← หายไปทั้งบรรทัด
 HOT-002   | ลดน้ำตาล 50% ได้    | อเมริกาโน่ร้อน — ลดน้ำตาล 50% ได้
 HOT-003   | [null]              | [null]
```

ผลลัพธ์เป็น `[null]` ถึง 13 จาก 20 แถว เพราะ 13 แถวนั้นไม่มีหมายเหตุ

> คิดแบบนี้: `NULL` แปลว่า "ไม่รู้ค่า"
> เอา `'ลาเต้ร้อน — '` ไปต่อกับ "ไม่รู้ค่า" ผลลัพธ์ก็ย่อมเป็น "ไม่รู้ค่า"
> **นี่คือกฎที่จะตามหลอกหลอนเราตลอดวันนี้: อะไรก็ตามที่แตะ `NULL` จะกลายเป็น `NULL`**

S7–S8 — `DISTINCT` และ `LIMIT` / `OFFSET`

```sql
SELECT DISTINCT category FROM menu;      -- 5 แถว
SELECT DISTINCT branch FROM sale_item;   -- 3 แถว
```

⚠️ **`DISTINCT` หลายคอลัมน์ = ตัดซ้ำจาก "คู่ของค่า" ไม่ใช่ตัดทีละคอลัมน์**

```sql
SELECT DISTINCT branch, pay_method FROM sale_item;
```

```
      branch       | pay_method
-------------------+------------
 สาขาสีลม          | เงินสด
 สาขาสีลม          | พร้อมเพย์
 สาขาสีลม          | บัตรเครดิต
 สาขารังสิต        | เงินสด
 ...                                  → รวม 9 แถว (3 สาขา × 3 วิธีชำระ)
```

`LIMIT` และ `OFFSET`

```sql
SELECT menu_name_th, price FROM menu LIMIT 5;              -- 5 แถวแรก
SELECT menu_name_th, price FROM menu LIMIT 5 OFFSET 5;     -- ข้าม 5 เอา 5 ถัดไป = "หน้า 2"
```

⚠️ **`LIMIT` ที่ไม่มี `ORDER BY` ไม่รับประกันว่าจะได้แถวไหน**

PostgreSQL มีสิทธิ์คืนแถวมาในลำดับใดก็ได้ และลำดับอาจเปลี่ยนเมื่อข้อมูลโตขึ้นหรือมีการสร้างดัชนี

ใช้ `LIMIT` เมื่อไร ให้ใส่ `ORDER BY` ด้วยเสมอ

ส่วนที่ 4

`WHERE` — กรองเอาเฉพาะแถวที่ต้องการ

ตัวดำเนินการใน `WHERE` ทั้งชุด

ตัวดำเนินการ	ความหมาย	ตัวอย่าง	ได้กี่แถว
`=`	เท่ากับ	`category = 'กาแฟร้อน'`	6
`<>` หรือ `!=`	ไม่เท่ากับ (ความหมายเดียวกัน)	`category <> 'ขนม'`	17
`>` `>=`	มากกว่า / มากกว่าหรือเท่ากับ	`price > 80`	8
`<` `<=`	น้อยกว่า / น้อยกว่าหรือเท่ากับ	`price <= 65`	5
`BETWEEN a AND b`	อยู่ในช่วง **รวมปลายทั้งสองข้าง**	`price BETWEEN 70 AND 90`	12
`IN (…)`	เป็นค่าใดค่าหนึ่งในรายการ	`category IN ('ชา','ขนม')`	7
`NOT IN (…)`	ไม่ใช่ค่าใด ๆ ในรายการ	`category NOT IN ('ชา','ขนม')`	13
`LIKE`	ตรงกับรูปแบบข้อความ	`menu_name_th LIKE 'ลา%'`	4
`IS NULL`	ไม่มีค่า	`kcal IS NULL`	2
`IS NOT NULL`	มีค่า	`email IS NOT NULL`	9

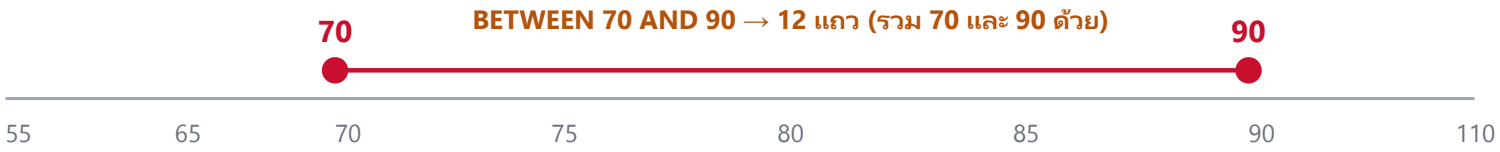
เชื่อมเงื่อนไขหลายอันด้วย `AND` · `OR` · `NOT`

> **ตัวเลขทางขวาคือคำตอบจริงของฐานข้อมูลเรา** — รันตามแล้วต้องได้เท่านี้ ถ้าไม่ตรงแปลว่าข้อมูลตั้งต้นผิด

W1–W7 — เปรียบเทียบค่า และการเลือกจากรายการ

```sql
SELECT menu_name_th, price FROM menu WHERE category = 'กาแฟร้อน';   -- 6 แถว
SELECT menu_name_th, price FROM menu WHERE price > 80;               -- 8 แถว
SELECT menu_name_th, price FROM menu WHERE price <= 65;              -- 5 แถว
SELECT menu_name_th, price FROM menu WHERE price < 65;               -- 3 แถว ← หายไป 2
```

`BETWEEN` รวมปลายทั้งสองข้าง




```sql
WHERE price BETWEEN 70 AND 90        -- 12 แถว
WHERE price >= 70 AND price <= 90    -- 12 แถวเท่ากัน — BETWEEN เป็นแค่ทางลัด
```

```sql
WHERE category IN ('ชา','ขนม')                    -- 7 แถว
WHERE category = 'ชา' OR category = 'ขนม'         -- 7 แถวเท่ากัน — IN คือทางลัดของ OR
WHERE category NOT IN ('ชา','ขนม')                -- 13 แถว
```

W8–W10 — `AND` · `OR` · คอลัมน์ boolean

```sql
-- AND : ต้องจริงทั้งคู่ → ผลลัพธ์ "แคบลง"
SELECT menu_name_th, category, price FROM menu
WHERE category = 'กาแฟเย็น' AND price >= 85;
-- 3 แถว : มอคค่าเย็น 85 · คาราเมลมัคคิอาโต้ 95 · ลาเต้มะพร้าวเย็น 95

-- OR : จริงข้อใดข้อหนึ่งก็พอ → ผลลัพธ์ "กว้างขึ้น"
SELECT menu_name_th, category, price FROM menu
WHERE category = 'ขนม' OR price > 90;
-- 5 แถว
```

ผลของ `OR` ข้างบนมาจากไหน

เข้าเงื่อนไข	เมนู
หมวดขนม	ครัวซองต์เนยสด · บราวนี่ช็อกโกแลต · **ชีสเค้กมะพร้าว**
ราคา > 90	คาราเมลมัคคิอาโต้ 95 · ลาเต้มะพร้าวเย็น 95 · **ชีสเค้กมะพร้าว 110**

ชีสเค้กมะพร้าวเข้าทั้งสองเงื่อนไข แต่ออกมา**แถวเดียว** — ได้ 3 + 3 − 1 = **5 แถว**

คอลัมน์ชนิด boolean ใช้ได้เลย

```sql
SELECT menu_name_th FROM menu WHERE is_signature;       -- 4 แถว (ไม่ต้องเขียน = true)
SELECT menu_name_th FROM menu WHERE NOT is_signature;   -- 16 แถว
```

⚠️ W11 — `AND` ทำงานก่อน `OR` เสมอ

คำเดียวกัน เรียงเหมือนกัน ต่างกันแค่วงเล็บ แต่คำตอบไม่เท่ากัน

```
-- แบบที่ 1 : ไม่ใส่วงเล็บ
WHERE category = 'กาแฟเย็น' AND price > 80 OR category = 'ขนม'
```

PostgreSQL อ่านเป็น `(category='กาแฟเย็น' AND price>80)` `OR` `category='ขนม'`

→ ได้กาแฟเย็นที่แพงกว่า 80 จำนวน 3 เมนู **บวก** ขนมทั้งหมด 3 เมนู = **6 แถว**

```
-- แบบที่ 2 : ใส่วงเล็บครอบ OR
WHERE category = 'กาแฟเย็น' AND (price > 80 OR category = 'ขนม')
```

คราวนี้ต้องเป็นกาแฟเย็นก่อนเสมอ ขนมจึงไม่มีทางหลุดเข้ามาได้ = **3 แถว**

แบบ	จำนวนแถว	ขนมติดมาด้วยไหม
ไม่ใส่วงเล็บ	**6**	ติดมาทั้ง 3 เมนู
ใส่วงเล็บครอบ `OR`	**3**	ไม่ติดเลย

> ✅ **กฎที่ควรติดตัวไปตลอด: เมื่อใดที่ `AND` กับ `OR` อยู่ในคำสั่งเดียวกัน ให้ใส่วงเล็บเสมอ**
> แม้วงเล็บนั้นจะไม่จำเป็นก็ควรใส่ เพื่อให้คนที่มาอ่านโค้ดของเราทีหลังเห็นเจตนาชัดเจน

W12 — `IS NULL` และกับดักที่เงียบที่สุดใน SQL

```
SELECT menu_name_th, kcal FROM menu WHERE kcal IS NULL;
```

```
   menu_name_th    |  kcal
-------------------+--------
 ลาเต้มะพร้าวเย็น     | [null]
 ชีสเค้กมะพร้าว       | [null]
                              → 2 แถว ✅
```

```
SELECT menu_name_th, kcal FROM menu WHERE kcal = NULL;
```

```
 menu_name_th | kcal
--------------+------
(0 rows)
                              → 0 แถว ⚠ และ "ไม่มี error" ด้วย
```

ทำไมถึงเป็นแบบนั้น

`NULL` แปลว่า **"ไม่รู้ค่า"** คำถามว่า *"ค่าที่ไม่รู้ เท่ากับ ค่าที่ไม่รู้ หรือไม่"* ตอบไม่ได้

PostgreSQL จึงตอบว่า **unknown** ซึ่ง `WHERE` ถือว่า**ไม่ผ่าน** → ตัดทิ้งทุกแถว

> **นี่คือบั๊กที่อันตรายที่สุดในบทนี้ เพราะมันไม่ฟ้อง error** โปรแกรมทำงานต่อได้ปกติ
> รายงานออกมาสวยงาม แค่ตัวเลขผิด — และไม่มีใครรู้จนกว่าจะมีคนไปนั่งนับเอง
> ✅ **กฎ: จะถาม `NULL` ต้องใช้ `IS NULL` / `IS NOT NULL` เท่านั้น ห้ามใช้ `=` หรือ `<>`**

⚠️⚠️ W13b — กับดักใหญ่ที่สุดของสัปดาห์นี้

ตาราง `customer` มี **12 แถว** และคอลัมน์ `email` เป็น `NULL` อยู่ 3 แถว

```
SELECT full_name, email FROM customer WHERE email LIKE '%@gmail.com';      -- 5 แถว
SELECT full_name, email FROM customer WHERE email NOT LIKE '%@gmail.com';  -- 4 แถว
```

LIKE '%@gmail.com'
5 แถว

NOT LIKE '%@gmail.com'
4 แถว

email IS NULL
3 แถว — ตกหล่นจากทั้งสองอัน

= 12

5 + 4 = 9 ไม่ใช่ 12 — อีก 3 แถวที่หายไปคือคนที่ไม่ได้ให้อีเมลไว้

พวกเขา **ไม่ผ่านทั้ง `LIKE` และ `NOT LIKE`** เพราะเปรียบเทียบกับค่าที่ไม่รู้ ผลลัพธ์ก็คือไม่รู้

```
-- ถ้าอยากได้ "ทุกคนที่ไม่ได้ใช้ gmail รวมคนที่ไม่ได้ให้อีเมลด้วย"
WHERE email NOT LIKE '%@gmail.com' OR email IS NULL      -- 7 แถว ✅
```

สไลด์นี้มีโอกาสสูงที่สุดที่จะออกสอบกลางภาค

เจอ `NOT` เมื่อไร ให้ถามตัวเองทันทีว่า "คอลัมน์นี้มี `NULL` ได้ไหม"

W14 — เปรียบเทียบวันที่และเวลา

วันที่และเวลาเปรียบเทียบด้วย `>` `<` `BETWEEN` ได้เหมือนตัวเลขทุกประการ

```
SELECT menu_name_th, launch_date FROM menu
WHERE  launch_date >= DATE '2026-01-01';
-- 3 แถว : ลาเต้น้ำผึ้งร้อน · ลาเต้มะพร้าวเย็น · ชีสเค้กมะพร้าว

SELECT menu_name_th, launch_date FROM menu
WHERE  launch_date BETWEEN DATE '2023-09-01' AND DATE '2024-12-31';
-- 7 แถว
```

⚠️ แต่กับ `timestamp` ต้องระวัง — ใช้ "ช่วงครึ่งเปิด"

```
-- ❌ วิธีที่ดูเหมือนถูก แต่ผิด
WHERE sold_at BETWEEN DATE '2026-09-18' AND DATE '2026-09-20'
-- DATE '2026-09-20' ถูกตีเป็นเวลา 00:00:00 ของวันนั้น
-- รายการที่ขายตอน 17:30 ของวันที่ 20 จะตกหล่นไปทั้งหมด

-- ✅ วิธีที่ถูก : >= วันเริ่ม  และ  < วันถัดไป
WHERE sold_at >= TIMESTAMP '2026-09-18 00:00:00'
  AND sold_at <  TIMESTAMP '2026-09-21 00:00:00'
-- 29 แถว (ยอดขายวันศุกร์ เสาร์ อาทิตย์ ครบถ้วน)
```

รูปแบบการเขียนค่า	ใช้กับ
`DATE 'YYYY-MM-DD'`	คอลัมน์ชนิด `date`
`TIMESTAMP 'YYYY-MM-DD HH:MM:SS'`	คอลัมน์ชนิด `timestamp`
`CURRENT_DATE` / `CURRENT_TIMESTAMP`	วันที่และเวลาปัจจุบันของเซิร์ฟเวอร์

? ชวนคิดข้อที่ 2 — ทายจำนวนแถว

ทั้ง 4 คำสั่งรันบนข้อมูลตั้งต้น (`menu` 20 แถว · `customer` 12 แถว)
ทายว่าแต่ละข้อได้กี่แถว แล้วค่อยรันเทียบ

```sql
-- (ก)
SELECT menu_name_th FROM menu WHERE price BETWEEN 65 AND 85;

-- (ข)
SELECT menu_name_th FROM menu
WHERE category = 'ชา' OR category = 'ขนม' AND price > 100;

-- (ค)
SELECT full_name FROM customer WHERE email IS NOT NULL AND tier = 'silver';

-- (ง)
SELECT menu_name_th FROM menu WHERE kcal > 200 OR kcal IS NULL;
```

ใบ้: ข้อ (ข) ไม่มีวงเล็บ · ข้อ (ง) มี `NULL` เข้ามาเกี่ยว

✅ แนวคำตอบข้อที่ 2

ข้อ	คำตอบ	เหตุผล
ก	**13 แถว**	ราคา 65(2) · 70(3) · 75(2) · 80(2) · 85(4) — `BETWEEN` รวมปลายทั้งสองข้าง
ข	**5 แถว**	`AND` ทำงานก่อน `OR` → อ่านเป็น `ชา OR (ขนม AND price>100)` = ชา 4 เมนู + ชีสเค้กมะพร้าว 110 อีก 1
ค	**4 แถว**	silver มี 6 คน แต่ M-003 และ M-009 ไม่ได้ให้อีเมลไว้ จึงถูกตัดออกทั้งคู่
ง	**11 แถว**	แคลอรีมากกว่า 200 มี 9 เมนู + ที่เป็น `NULL` อีก 2 = 11

บทเรียนจากข้อนี้

1. **ข้อ (ข)** ถ้าใส่วงเล็บเป็น `(ชา OR ขนม) AND price > 100` จะได้แค่ **1 แถว** — จาก 5 เหลือ 1
   คำเหมือนกันทุกตัว ต่างแค่วงเล็บ แต่คำตอบต่างกัน 5 เท่า
2. **ข้อ (ค)** `AND` เจอ `NULL` เมื่อไร แถวนั้นหายเสมอ แม้เงื่อนไขอีกข้างจะจริงก็ตาม
   ระวังให้ดี: `tier = 'silver'` เป็นจริงทั้ง 6 คน แต่ได้ผลลัพธ์แค่ 4
3. **ข้อ (ง)** `kcal > 200` อย่างเดียวจะได้ 9 แถว — แถว `NULL` ไม่ติดมาเอง
   ถ้าอยากให้ `NULL` ติดมาด้วย ต้องเขียน `OR ... IS NULL` ต่อท้ายเสมอ ไม่มีทางลัด
   (และสังเกตว่า `TEA-002` ที่มี 200 พอดีไม่ติดมา เพราะเราเขียน `>` ไม่ใช่ `>=`)

ส่วนที่ 5

`LIKE` — ค้นหาข้อความแบบ "คล้าย ๆ"

ไฟล์ `04-like.sql`

ไวยากรณ์ `LIKE` — จำแค่ 2 สัญลักษณ์

สัญลักษณ์	แทนอะไร
`%`	อักขระ **กี่ตัวก็ได้** รวมถึง **ศูนย์ตัว**
`_`	อักขระ **หนึ่งตัวพอดี**

รูปแบบ	อ่านว่า	ตัวอย่างที่ตรง	ได้กี่แถว
`'ลา%'`	ขึ้นต้นด้วย ลา	ลาเต้ร้อน · ลาเต้เย็น	4
`'%เย็น'`	ลงท้ายด้วย เย็น	ชาไทยเย็น · โกโก้เย็น	6
`'%ลาเต้%'`	มี ลาเต้ อยู่ตรงไหนก็ได้	ลาเต้ร้อน · ชาเขียวมัทฉะลาเต้	5
`'HOT-00_'`	HOT-00 แล้วต่ออีก 1 ตัว	HOT-001 … HOT-006	6
`'___-001'`	อะไรก็ได้ 3 ตัว แล้ว -001	HOT-001 · ICE-001 · TEA-001	5
`'ลาเต้'`	ต้องเป็นคำนี้เป๊ะ ๆ	*(ไม่มีเมนูไหนชื่อ "ลาเต้" เฉย ๆ)*	0

⚠️ **`LIKE` ที่ไม่มี `%` และไม่มี `_` เลย มีค่าเท่ากับเครื่องหมาย `=` ทุกประการ**

เขียน `WHERE menu_name_th LIKE 'ลาเต้ร้อน'` กับ `WHERE menu_name_th = 'ลาเต้ร้อน'` ได้ผลเหมือนกัน

L1–L3 — ขึ้นต้น / ลงท้าย / มีอยู่ภายใน

```
SELECT menu_code, menu_name_th FROM menu WHERE menu_name_th LIKE 'ลา%';
```

```
 HOT-003  ลาเต้ร้อน           HOT-006  ลาเต้น้ำผึ้งร้อน
 ICE-002  ลาเต้เย็น           ICE-005  ลาเต้มะพร้าวเย็น          → 4 แถว
```

```
SELECT menu_code, menu_name_th FROM menu WHERE menu_name_th LIKE '%ลาเต้%';
```

```
 HOT-003  ลาเต้ร้อน           HOT-006  ลาเต้น้ำผึ้งร้อน
 ICE-002  ลาเต้เย็น           ICE-005  ลาเต้มะพร้าวเย็น
 TEA-002  ชาเขียวมัทฉะลาเต้   ← แถวที่เพิ่มเข้ามา!            → 5 แถว
```

ต่างกันแค่ `%` ตัวเดียวข้างหน้า แต่ได้ผลไม่เท่ากัน

`ชาเขียวมัทฉะลาเต้` มีคำว่า "ลาเต้" อยู่จริง แต่**ไม่ได้ขึ้นต้น**ด้วยคำนั้น

`'ลา%'` จึงไม่จับ ส่วน `'%ลาเต้%'` จับได้

> ✅ **ถ้าลูกค้าเดินเข้าร้านแล้วถามว่า "มีลาเต้อะไรบ้าง" คำตอบที่ถูกคือ `'%ลาเต้%'`**
>
> อีกตัวอย่าง: `'%เย็น'` ได้ 6 แถว แต่ **ไม่รวม** คาราเมลมัคคิอาโต้ ทั้งที่เป็นกาแฟเย็น
>
> เพราะ `LIKE` ดูที่ **ตัวอักษร** ไม่ได้ดูที่ **ความหมาย**

L4–L6 — `_` นับทีละตัว และ `NOT LIKE`

```
SELECT menu_code, menu_name_th FROM menu WHERE menu_code LIKE 'HOT-00_';
-- 6 แถว : HOT-001 ถึง HOT-006

SELECT menu_code, menu_name_th, category FROM menu WHERE menu_code LIKE '___-001';
-- 5 แถว : HOT-001 · ICE-001 · TEA-001 · BAK-001 · OTH-001  (สินค้าตัวแรกของทุกหมวด)
```

`_` ผสมกับข้อความคงที่ — รูปแบบเดียวกับที่เราใส่ใน `CHECK` ตั้งแต่ไฟล์ `01`

```
SELECT full_name, phone FROM customer WHERE phone LIKE '0__-___-____';
-- 12 แถว (ทุกคน เพราะ CHECK บังคับรูปแบบนี้ไว้ตั้งแต่ตอน INSERT)

SELECT full_name, phone FROM customer WHERE phone LIKE '08_-234%';
-- 3 แถว : สมชาย ใจดี · สมหญิง รักเรียน · สมศักดิ์ ตั้งใจ
```

`NOT LIKE`

```
SELECT menu_name_en FROM menu WHERE menu_name_en LIKE '%Latte%';      -- 5 แถว
SELECT menu_name_en FROM menu WHERE menu_name_en NOT LIKE '%Latte%';  -- 15 แถว
```

> 5 + 15 = 20 พอดี **เพราะ** `menu_name_en` **เป็น** `NOT NULL` **ทุกแถว**
> ถ้าเป็นคอลัมน์ที่ `NULL` ได้ ตัวเลขจะไม่บวกกันแบบนี้ — เดี๋ยวจะเห็นในสไลด์ถัดไปอีก 3 หน้า

L7–L8 — `LIKE` แยกตัวพิมพ์ แต่ `ILIKE` ไม่แยก

```
SELECT menu_name_en FROM menu WHERE menu_name_en LIKE '%latte%';
```

```
 menu_name_en
--------------
(0 rows)                    ← 0 แถว ⚠
```

ในฐานข้อมูลเขียนว่า `Latte` ตัว L ใหญ่ แต่เราค้นด้วย `latte` ตัว l เล็ก

```
SELECT menu_name_en FROM menu WHERE menu_name_en ILIKE '%latte%';
```

```
 Hot Latte · Hot Honey Latte · Iced Latte
 Iced Coconut Latte · Matcha Green Tea Latte        → 5 แถว ✅
```

คำสั่ง	แยกตัวพิมพ์ไหม	เป็นมาตรฐาน SQL ไหม
`LIKE`	แยก	✅ ใช้ได้ทุกฐานข้อมูล
`ILIKE`	ไม่แยก	❌ **เฉพาะ PostgreSQL**
`LOWER(คอลัมน์) LIKE '%latte%'`	ไม่แยก	✅ ยกไปใช้ที่อื่นได้

> **นี่คือสาเหตุอันดับหนึ่งที่นักศึกษาคิดว่า "ค้นแล้วไม่เจอ ข้อมูลคงหาย"**
> ถ้าระบบที่เขียนอาจต้องย้ายไป MySQL หรือ SQL Server ในอนาคต ให้ใช้ `LOWER(...)` แทน `ILIKE`

L9 — `LIKE` กับข้อความภาษาไทย

```
SELECT menu_name_th FROM menu WHERE menu_name_th LIKE  '%ลาเต้%';   -- 5 แถว
SELECT menu_name_th FROM menu WHERE menu_name_th ILIKE '%ลาเต้%';   -- 5 แถวเท่ากัน
```

ภาษาไทยไม่มีตัวพิมพ์ใหญ่-เล็ก `LIKE` กับ `ILIKE` จึงให้ผลเหมือนกันเสมอ

```
SELECT full_name FROM customer WHERE full_name LIKE 'สม%';    -- 3 แถว
SELECT full_name FROM customer WHERE full_name LIKE 'ณัฐ%';   -- 2 แถว
SELECT full_name FROM customer WHERE full_name LIKE '%ทอง%';  -- 2 แถว
```

⚠️ แต่มีเรื่องที่ต้องระวังมาก — อย่าใช้ `_` นับตัวอักษรไทย

สระและวรรณยุกต์ไทยถูกเก็บเป็น **อักขระแยกต่างหาก** จากพยัญชนะ

คำ	คนไทยเห็นกี่ตัว	คอมพิวเตอร์นับกี่อักขระ
ชา	2	2 (ช + า)
ชื่อ	3	**4** (ช + ื + ่ + อ)
เต้	3	**3** (เ + ต + ้)

> `LIKE 'ช_'` จึงไม่ได้แปลว่า "ช แล้วตามด้วยตัวอักษรไทยอีก 1 ตัวที่เรามองเห็น"
>
> ✅ **สรุป: ใช้ `%` กับภาษาไทยได้สบาย แต่อย่าใช้ `_` นับตัวอักษรไทย**

L10 — `ESCAPE` เมื่อ `%` เป็นตัวอักษรจริง

โจทย์: อยากหาเมนูที่หมายเหตุพูดถึงส่วนลด "50%"

ในตาราง `menu` มีหมายเหตุอยู่ 7 แถว สองแถวนี้เกี่ยวข้อง

menu_code	note
HOT-002	ลดน้ำตาล **50%** ได้
ICE-004	ขายดีอันดับ 1 เฉลี่ย 1**50** แก้วต่อวัน

```
-- ❌ วิธีที่คนส่วนใหญ่เขียนครั้งแรก
SELECT menu_code, note FROM menu WHERE note LIKE '%50%%';     -- ได้ 2 แถว (เกินมา 1)
```

`%` ตัวสุดท้ายถูกตีความเป็น "อะไรก็ได้" ไม่ใช่เครื่องหมายเปอร์เซ็นต์

คำสั่งนี้จึงแปลว่า *"หาข้อความที่มีเลข 50 อยู่"* ซึ่ง "150" ก็เข้าข่าย

```
-- ✅ วิธีที่ถูก
SELECT menu_code, note FROM menu WHERE note LIKE '%50#%%' ESCAPE '#';    -- ได้ 1 แถว
```

ส่วนของรูปแบบ	ความหมาย
`%`	อะไรก็ได้
`50`	ต้องเป็นเลข 50 จริง
`#%`	`#` คือตัวหนีที่เราประกาศไว้ → ตัวถัดจาก `#` กลายเป็นอักขระธรรมดา = `%` ตัวจริง
`%`	อะไรก็ได้

L11–L13 — `LIKE` ผสมเงื่อนไขอื่น และ `LIKE` กับ `NULL`

```sql
SELECT menu_code, menu_name_th, price FROM menu
WHERE  category = 'กาแฟเย็น' AND menu_name_th LIKE '%ลาเต้%';
-- 2 แถว : ลาเต้เย็น · ลาเต้มะพร้าวเย็น
-- (ลาเต้ร้อนกับลาเต้น้ำผึ้งร้อนตกไป เพราะอยู่หมวดกาแฟร้อน)

SELECT menu_code, menu_name_th, menu_name_en FROM menu
WHERE  menu_name_th LIKE '%มะพร้าว%' OR menu_name_en ILIKE '%coconut%';
-- 2 แถว : ลาเต้มะพร้าวเย็น · ชีสเค้กมะพร้าว
```

⚠️ L13 — ตัวเลขที่ไม่ยอมบวกกัน (อีกครั้ง)

```sql
SELECT COUNT(*) FROM sale_item;                                  -- 60
SELECT COUNT(*) FROM sale_item WHERE remark LIKE '%หวาน%';       --  2
SELECT COUNT(*) FROM sale_item WHERE remark NOT LIKE '%หวาน%';   --  9
```

2 + 9 = 11 ไม่ใช่ 60 เพราะ `remark` เป็น `NULL` อยู่ **49 แถว**

```sql
SELECT COUNT(*) FROM sale_item WHERE remark NOT LIKE '%หวาน%' OR remark IS NULL;   -- 58 ✅
```

> เจอกับดักนี้เป็นครั้งที่ 2 แล้ววันนี้ (ครั้งแรกคือเรื่อง gmail)
> **ทุกครั้งที่เขียน `NOT` บนคอลัมน์ที่เป็น `NULL` ได้ ให้คิดถึง `OR ... IS NULL` ไว้เสมอ**

? ชวนคิดข้อที่ 3 — จับคู่รูปแบบกับจำนวนแถว

รายชื่อเมนูทั้ง 20 อยู่ในไฟล์ `02-insert.sql`

จับคู่รูปแบบทางซ้ายกับจำนวนแถวทางขวา (จำนวนแถวมีให้เกิน ใช้ไม่ครบ)

#	รูปแบบ
1	`menu_name_th LIKE 'ชา%'`
2	`menu_name_th LIKE '%ร้อน'`
3	`menu_code LIKE '%-00_'`
4	`menu_name_en LIKE '%Iced%'`
5	`menu_name_en ILIKE 'iced%'`
6	`menu_name_th LIKE '%มอคค่า%'`

ตัวเลือก: ก) 0 · ข) 2 · ค) 4 · ง) 5 · จ) 6 · ฉ) 7 · ช) 20

ใบ้: ข้อ 2 อย่าเพิ่งตอบ 6 — ไปดูชื่อเมนูกาแฟร้อนให้ครบทั้ง 6 ตัวก่อน
ข้อ 3 ให้ดูว่ามีรหัสไหน**ไม่**เข้ารูปแบบบ้าง · ข้อ 5 สังเกตว่าไม่มี `%` ข้างหน้า

✅ แนวคำตอบข้อที่ 3

#	รูปแบบ	คำตอบ	เมนูที่ตรง
1	`'ชา%'`	**ค) 4**	ชาไทยเย็น · ชาเขียวมัทฉะลาเต้ · ชามะนาว · ชานมไข่มุก
2	`'%ร้อน'`	**ง) 5**	อเมริกาโน่ร้อน · ลาเต้ร้อน · คาปูชิโน่ร้อน · มอคค่าร้อน · ลาเต้น้ำผึ้งร้อน
3	`'%-00_'`	**ช) 20**	รหัสทุกตัวมีรูปแบบ `XXX-00N` — `%` กินสามตัวหน้าได้หมด
4	`'%Iced%'`	**ฉ) 7**	Iced Americano/Latte/Mocha/Coconut Latte/Cocoa + **Thai Iced Tea · Lemon Iced Tea**
5	`ILIKE 'iced%'`	**ง) 5**	ไม่มี `%` ข้างหน้า = ต้อง**ขึ้นต้น**ด้วย iced → หายไป 2 ตัวที่ Iced อยู่กลางคำ
6	`'%มอคค่า%'`	**ข) 2**	มอคค่าร้อน · มอคค่าเย็น

สามเรื่องที่ควรเก็บกลับไป

1. **ข้อ 2 หลอก** — หมวดกาแฟร้อนมี 6 เมนู แต่ `เอสเพรสโซ่` ไม่ได้ลงท้ายด้วย "ร้อน"
   `LIKE` **ดูที่ตัวอักษร ไม่ได้ดูที่หมวดหมู่หรือความหมาย**
2. **ข้อ 3** `%` กินอะไรก็ได้ รวมถึงหลายตัวอักษร — คนมักคิดว่า `%` แทนได้ตัวเดียว
   (ตัวที่แทนได้ตัวเดียวคือ `_`)
3. **ข้อ 4 กับ 5 ต่างกันถึง 2 แถว** ทั้งที่ดูเผิน ๆ เหมือนถามเรื่องเดียวกัน
   `'%Iced%'` = "มีคำนี้อยู่" ส่วน `'iced%'` = "ขึ้นต้นด้วยคำนี้"
   → **เขียน pattern ให้อ่านจากซ้ายไปขวา แล้วถามทีละตัวว่า "ตรงนี้บังคับอะไร"**

ส่วนที่ 6

`ORDER BY` — เรียงลำดับผลลัพธ์

ไฟล์ `05-order-by.sql`

01–03 — `ASC` / `DESC` และปัญหาของค่าซ้ำ

```
SELECT menu_name_th, price FROM menu ORDER BY price;        -- ASC เป็นค่าตั้งต้น
SELECT menu_name_th, price FROM menu ORDER BY price DESC;
```

ปัญหา: ในตารางเรามีเมนูราคา 85 บาทถึง 4 รายการ

```
SELECT menu_name_th, category, price FROM menu WHERE price = 85;
```

```
ลาเต้น้ำผึ้งร้อน   กาแฟร้อน   85.00      มอคค่าเย็น        กาแฟเย็น   85.00
ชานมไข่มุก        ชา         85.00      บราวนี่ช็อกโกแลต   ขนม       85.00
```

เรียงด้วยคีย์เดียว สี่แถวนี้จะออกมา**ลำดับไหนก็ได้** และอาจเปลี่ยนในการรันครั้งถัดไป

```
-- ✅ ทางแก้ : เพิ่มคีย์รอง
SELECT menu_name_th, category, price FROM menu
ORDER BY price DESC, menu_name_th ASC;
```

> ✅ **กฎที่ควรติดตัว: ถ้าคอลัมน์ที่ใช้เรียงมีค่าซ้ำได้ ให้ใส่คีย์รองเสมอ**
> คีย์รองที่ปลอดภัยที่สุดคือ**คีย์หลักของตาราง** เพราะไม่มีทางซ้ำแน่นอน
> ปัญหานี้จะหนักขึ้นมากเมื่อใช้ร่วมกับ `LIMIT` ในหน้าถัด ๆ ไป

04 — `ORDER BY` ใช้ชื่อแทนได้ แต่ `WHERE` ใช้ไม่ได้

```sql
SELECT menu_name_th, price, cost, price - cost AS กำไร
FROM   menu
ORDER  BY กำไร DESC;        -- ✅ ใช้ได้
```

```
   menu_name_th    | price | cost  | กำไร
-------------------+-------+-------+-------
 ชีสเค้กมะพร้าว      |110.00 | 45.00 | 65.00
 คาราเมลมัคคิอาโต้   | 95.00 | 38.00 | 57.00
 ลาเต้มะพร้าวเย็น    | 95.00 | 40.00 | 55.00
```

```sql
SELECT menu_name_th, price - cost AS กำไร
FROM   menu
WHERE  กำไร > 50;           -- ❌ ERROR: column "กำไร" does not exist  (42703)
```

ทำไมอันหนึ่งได้ อีกอันไม่ได้ — กลับไปดูแผนภาพลำดับการทำงาน

```
FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT
        ↑                                  ↑          ↑
   ยังไม่ถึง SELECT              ชื่อ "กำไร"     ใช้ชื่อได้แล้ว
   ชื่อ "กำไร" ยังไม่เกิด       เพิ่งถูกสร้าง
```

```sql
-- ✅ ถ้าอยากกรองด้วยนิพจน์ ต้องเขียนนิพจน์ซ้ำใน WHERE
SELECT menu_name_th, price - cost AS กำไร FROM menu
WHERE price - cost > 50 ORDER BY กำไร DESC;    -- 5 แถว
```

> นี่คือครั้งแรกที่แผนภาพลำดับการทำงานมีประโยชน์จริง — และจะไม่ใช่ครั้งสุดท้าย

O5–O6 — ตามลำดับคอลัมน์ และตามนิพจน์

```
SELECT menu_name_th, category, price FROM menu ORDER BY 3 DESC, 1 ASC;
```

เลข `3` = คอลัมน์ที่สามใน `SELECT` (`price`) · เลข `1` = คอลัมน์แรก (`menu_name_th`)

> ⚠️ **เปราะบางมาก** ถ้ามีคนสลับลำดับคอลัมน์ใน `SELECT` ทีหลัง
> การเรียงจะเปลี่ยนไปเงียบ ๆ โดยไม่มี error ให้เห็น — **เขียนชื่อคอลัมน์ไปเลยปลอดภัยกว่า**

`ORDER BY` ตามนิพจน์ที่คำนวณสด

```
SELECT receipt_no, menu_name_th, qty, unit_price, discount,
       qty * unit_price - discount AS ยอดรายการ
FROM   sale_item
ORDER  BY qty * unit_price - discount DESC
LIMIT  10;
```

```
 receipt_no |    menu_name_th     | qty | unit_price | discount | ยอดรายการ
------------+---------------------+-----+------------+----------+-----------
 R69-0029   | คาราเมลมัคคิอาโต้   |   3 |      95.00 |    15.00 |    270.00
 R69-0019   | ลาเต้เย็น           |   3 |      75.00 |     0.00 |    225.00
 R69-0014   | คาราเมลมัคคิอาโต้   |   2 |      95.00 |     0.00 |    190.00
 ...                                                          → 10 แถว
```

```
-- เขียนสั้นกว่าด้วยชื่อแทน ได้ผลเหมือนกันเป๊ะ
ORDER BY ยอดรายการ DESC LIMIT 10;
```

07 — `NULL` ไปอยู่ตรงไหนของลำดับ

สั่งอย่างไร	`NULL` ไปอยู่ตรงไหน
`ORDER BY kcal` (ASC)	**ท้ายสุด** — ค่าตั้งต้นของ PostgreSQL คือ `NULLS LAST`
`ORDER BY kcal DESC`	**บนสุด** — ค่าตั้งต้นของ DESC คือ `NULLS FIRST`
`ORDER BY kcal ASC NULLS FIRST`	บนสุด (สั่งเอง)
`ORDER BY kcal DESC NULLS LAST`	ท้ายสุด (สั่งเอง)

```sql
SELECT menu_name_th, kcal FROM menu ORDER BY kcal DESC;
```

```
   menu_name_th    |  kcal
-------------------+--------
 ลาเต้มะพร้าวเย็น    | [null]   ← โผล่มาบนสุดทั้งที่เราขอ "แคลอรีมากไปน้อย"
 ชีสเค้กมะพร้าว      | [null]
 บราวนี่ช็อกโกแลต    |    420
 ชานมไข่มุก          |    350
```

```sql
SELECT menu_name_th, kcal FROM menu ORDER BY kcal DESC NULLS LAST;   -- ✅ เริ่มที่ 420
```

เวลาทำรายงานให้ผู้บริหารดู แทบไม่มีใครอยากให้แถวว่างโผล่มาอยู่บนสุด

✅ **เขียน `NULLS LAST` ให้ชัดเจนเสมอ อย่าพึ่งค่าตั้งต้น** เพราะฐานข้อมูลยี่ห้ออื่นตั้งต้นไม่เหมือนกัน

O8–O9 — Top-N : คำสั่งที่ใช้บ่อยที่สุดในการทำงานจริง

```sql
-- "5 เมนูที่แพงที่สุด"
SELECT menu_name_th, price FROM menu
ORDER BY price DESC, menu_name_th ASC
LIMIT 5;
```

```
   menu_name_th    | price
-------------------+--------
 ชีสเค้กมะพร้าว      | 110.00
 คาราเมลมัคคิอาโต้   |  95.00
 ลาเต้มะพร้าวเย็น    |  95.00
 ชาเขียวมัทฉะลาเต้   |  90.00
 ชานมไข่มุก          |  85.00    ← ถ้าไม่ใส่คีย์รอง แถวนี้จะเป็นอะไรก็ได้ใน 4 เมนูราคา 85
```

```sql
-- "10 รายการขายล่าสุดของสาขาสีลม"
SELECT receipt_no, sold_at, menu_name_th, qty FROM sale_item
WHERE  branch = 'สาขาสีลม'
ORDER  BY sold_at DESC
LIMIT  10;
```

ลำดับที่เกิดขึ้นจริง

① `WHERE` กรองเหลือ 29 แถว → ② `ORDER BY` เรียง 29 แถวนั้น → ③ `LIMIT` ตัดเหลือ 10

> ถ้าสลับลำดับเป็น "ตัด 10 แถวก่อน แล้วค่อยเรียง" คำตอบจะผิดทันที
> **ความหมายของ Top-N อยู่ที่ลำดับการทำงาน ไม่ใช่แค่ไวยากรณ์**

⚠️ O10 — เรียงข้อความภาษาไทยแล้วผลไม่ตรงกับที่คาด

```
SELECT menu_name_th FROM menu ORDER BY menu_name_th COLLATE "C";
```

```
... (17 แถวแรก เรียงตามปกติ ค → ช → น → บ → ม → ล → อ)
18. อเมริกาโน่เย็น
19. เอสเพรสโซ่          ← ทำไมอยู่ท้าย?
20. โกโก้เย็น           ← ทำไม "ก" อยู่ท้ายสุด?
```

เพราะสระหน้ามีรหัสอักขระสูงกว่าพยัญชนะทุกตัว

อักขระ	รหัส Unicode	เลขฐานสิบ
`ก` (พยัญชนะตัวแรก)	U+0E01	3585
`ฮ` (พยัญชนะตัวสุดท้าย)	U+0E2E	3630
`เ` `แ` `โ` `ใ` `ไ` (สระหน้า)	U+0E40 – U+0E44	3648–3652

คอมพิวเตอร์เรียงตาม**รหัส** ไม่ได้เรียงตามวิธีที่เราเปิดพจนานุกรม

```
-- ✅ ทางแก้ (ถ้าเซิร์ฟเวอร์ติดตั้ง ICU มาด้วย)
SELECT menu_name_th FROM menu ORDER BY menu_name_th COLLATE "th-TH-x-icu";

-- ตรวจก่อนว่าเครื่องนี้รองรับไหม
SELECT collname FROM pg_collation WHERE collname LIKE 'th%';
```

✅ **สิ่งที่ต้องจำไปทำงาน: การเรียงข้อความไม่ได้ขึ้นกับ `ORDER BY` อย่างเดียว**

แต่ขึ้นกับ "กฎการเรียง (collation)" ของฐานข้อมูลด้วย — ออกแบบระบบภาษาไทยต้องคุยเรื่องนี้ตั้งแต่วันแรก

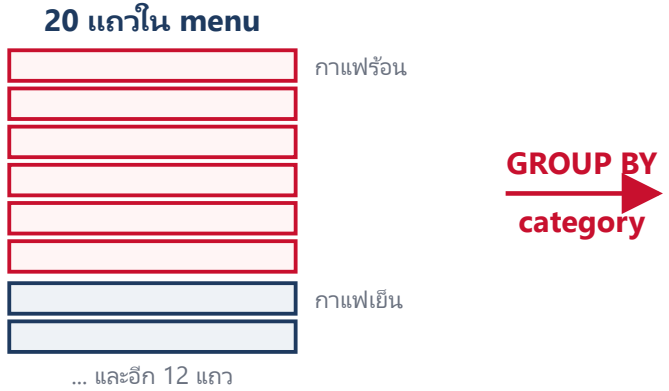
ส่วนที่ 7

`GROUP BY` — สรุปยอดเป็นกลุ่ม

ไฟล์ `06-group-by.sql`

แนวคิดหลักมีประโยคเดียว

GROUP BY คือการยุบหลายแถวให้เหลือ หนึ่งแถวต่อหนึ่งกลุ่ม



5 แถวออก

กาแฟร้อน · 6 เมนู · เฉลี่ย 70.00 · 55–85

กาแฟเย็น · 5 เมนู · เฉลี่ย 83.00 · 65–95

ชา · 4 เมนู · เฉลี่ย 75.00 · 60–90

ขนม · 3 เมนู · เฉลี่ย 90.00 · 75–110

G1 — ฟังก์ชันรวมที่ยังไม่มี `GROUP BY`

ทั้งตารางถือเป็นกลุ่มเดียว จึงได้ผลลัพธ์แถวเดียวเสมอ

```
SELECT COUNT(*) FROM sale_item;
```

```
 count
-------
    60          ← ตารางมี 60 แถว แต่ผลลัพธ์ออกมาแถวเดียว
```

```
SELECT COUNT(*)   AS จำนวนเมนู,
       SUM(price) AS รวมราคาทุกเมนู,
       AVG(price) AS ราคาเฉลี่ย,
       MIN(price) AS ถูกที่สุด,
       MAX(price) AS แพงที่สุด
FROM   menu;
```

```
 จำนวนเมนู | รวมราคาทุกเมนู |       ราคาเฉลี่ย        | ถูกที่สุด | แพงที่สุด
-----------+----------------+-------------------------+----------+----------
        20 |        1555.00 | 77.7500000000000000     |    55.00 |   110.00
```

เพราะเราสั่งให้ **"สรุป"** ไม่ได้สั่งให้ **"แสดง"**

⚠️ แต่ค่าเฉลี่ยออกมาทศนิยมยาวเฟื้อยอ่านไม่รู้เรื่อง — แก้ในหน้าถัดไป

G2 — ⚠️ `COUNT(*)` ไม่เท่ากับ `COUNT(ชื่อคอลัมน์)`

```
SELECT COUNT(*)    AS ทุกแถว,
       COUNT(kcal) AS มีค่าแคลอรี,
       COUNT(note) AS มีหมายเหตุ
FROM   menu;
```

```
 ทุกแถว | มีค่าแคลอรี | มีหมายเหตุ
--------+-------------+------------
     20 |          18 |          7
```

เขียนอย่างไร	นับอะไร
`COUNT(*)`	**ทุกแถว** ไม่ว่าจะมีค่าหรือไม่
`COUNT(คอลัมน์)`	**เฉพาะแถวที่คอลัมน์นั้นมีค่า** (ไม่เป็น `NULL`)

```
SELECT COUNT(*) AS ทุกแถว, COUNT(email) AS มีอีเมล FROM customer;
--  12 | 9

SELECT COUNT(*) AS ทุกแถว, COUNT(member_code) AS เป็นสมาชิก,
       COUNT(remark) AS มีหมายเหตุ FROM sale_item;
--  60 | 31 | 11
```

บทเรียนทั้งหมดเรื่อง `NULL` สรุปได้ในผลลัพธ์แถวเดียวนี้

`20 | 18 | 7` บอกทันทีว่าเมนู 2 รายการยังไม่ได้วัดแคลอรี และ 13 รายการไม่มีหมายเหตุ
เทคนิคนี้ใช้ตรวจคุณภาพข้อมูลได้เร็วมาก — เจอตารางใหม่ ให้ลองนับแบบนี้ก่อนเสมอ

G5 — `ROUND` กับ `AVG` และ `NULL` ที่หลบอยู่ในตัวหาร

```sql
SELECT ROUND(AVG(price), 2) AS ราคาเฉลี่ย,
       ROUND(AVG(cost),  2) AS ต้นทุนเฉลี่ย,
       ROUND(AVG(price - cost), 2) AS กำไรเฉลี่ย
FROM   menu;
```

```
 ราคาเฉลี่ย | ต้นทุนเฉลี่ย | กำไรเฉลี่ย
------------+--------------+------------
      77.75 |        29.65 |      48.10
```

⚠️ แต่กับคอลัมน์ที่มี `NULL` ต้องระวัง

```sql
SELECT COUNT(*) AS จำนวนเมนูทั้งหมด, COUNT(kcal) AS วัดแคลอรีแล้ว,
       SUM(kcal) AS รวมแคลอรี, ROUND(AVG(kcal), 1) AS แคลอรีเฉลี่ย
FROM   menu;
```

```
 จำนวนเมนูทั้งหมด | วัดแคลอรีแล้ว | รวมแคลอรี | แคลอรีเฉลี่ย
------------------+---------------+-----------+--------------
                20 |            18 |      3565 |        198.1
```

3565 ÷ 20 = 178.25 แต่คำตอบที่ได้คือ 198.1 เพราะ `AVG` หารด้วย **18** ไม่ใช่ 20

แถวที่เป็น `NULL` ถูกตัดออกจาก**ทั้งตัวตั้งและตัวหาร**

> ✅ **ทุกครั้งที่เห็น `AVG` ให้ถามตัวเองทันทีว่า "คอลัมน์นี้มี `NULL` ไหม"**
>
> วิธีตรวจที่เร็วที่สุดคือเทียบ `COUNT(คอลัมน์)` กับ `COUNT(*)` — ถ้าไม่เท่ากัน แปลว่ามี `NULL`

G6–G7 — `GROUP BY` คอลัมน์เดียว (สไลด์หลักของบทนี้)

```sql
SELECT category                AS หมวด,
       COUNT(*)                AS จำนวนเมนู,
       ROUND(AVG(price), 2)    AS ราคาเฉลี่ย,
       MIN(price)              AS ถูกสุด,
       MAX(price)              AS แพงสุด,
       MAX(price) - MIN(price) AS ช่วงราคา
FROM   menu
GROUP  BY category
ORDER  BY ราคาเฉลี่ย DESC;
```

```
      หมวด       | จำนวนเมนู | ราคาเฉลี่ย | ถูกสุด | แพงสุด | ช่วงราคา
-----------------+-----------+------------+--------+--------+----------
 ขนม             |         3 |      90.00 |  75.00 | 110.00 |    35.00
 กาแฟเย็น        |         5 |      83.00 |  65.00 |  95.00 |    30.00
 ชา              |         4 |      75.00 |  60.00 |  90.00 |    30.00
 เครื่องดื่มอื่น    |         2 |      75.00 |  70.00 |  80.00 |    10.00
 กาแฟร้อน        |         6 |      70.00 |  55.00 |  85.00 |    30.00
                       ↑ รวม 20 ✓
```

✅ **กฎเหล็กข้อที่ 1: คอลัมน์ใดที่อยู่ใน `SELECT` และไม่ได้ถูกห่อด้วยฟังก์ชันรวม ต้องปรากฏใน `GROUP BY` ด้วยเสมอ**

⚠️ สังเกต ชา กับ เครื่องดื่มอื่น ราคาเฉลี่ยเท่ากันที่ 75.00 — **ปัญหาค่าซ้ำจากบท `ORDER BY` ตามมาหลอกอีกแล้ว**

ถ้าอยากได้ลำดับคงที่ ต้องเพิ่ม `, หมวด` ต่อท้าย `ORDER BY`

G8 — `SUM` ของนิพจน์ = คำถามที่เจ้าของร้านอยากรู้จริง ๆ

```sql
SELECT category AS หมวด,
       COUNT(*) AS จำนวนรายการ,
       SUM(qty) AS แก้วที่ขายได้,
       SUM(qty * unit_price - discount) AS ยอดขาย
FROM   sale_item
GROUP  BY category
ORDER  BY ยอดขาย DESC;
```

```
      หมวด       | จำนวนรายการ | แก้วที่ขายได้ | ยอดขาย
-----------------+-------------+---------------+----------
 กาแฟเย็น        |          19 |            29 |  2385.00
 กาแฟร้อน        |          17 |            21 |  1440.00
 ชา              |          11 |            18 |  1330.00
 ขนม             |           8 |            10 |   815.00
 เครื่องดื่มอื่น    |           5 |             5 |   380.00
-----------------+-------------+---------------+----------
 รวมทั้งร้าน      |          60 |            83 |  6350.00
```

ตัวเลข 3 ชุดนี้ตอบคำถามคนละข้อ

ตัวเลข	ตอบคำถามว่า
`COUNT(*)`	ขายไปกี่**ครั้ง** (มีลูกค้าสั่งกี่บรรทัด)
`SUM(qty)`	ขายไปกี่**แก้ว**
`SUM(qty * unit_price - discount)`	ได้**เงิน**เท่าไร

และเรียงลำดับได้ไม่เหมือนกันด้วย — กาแฟร้อนขายบ่อยเป็นอันดับ 2 แต่ได้เงินน้อยกว่ากาแฟเย็นเกือบเท่าตัว เพราะกาแฟเย็นราคาสูงกว่าและลูกค้าสั่งทีละหลายแก้ว **นี่คือเหตุผลที่ต้องสรุปหลายมุม ไม่ใช่มุมเดียว**

G9 — `GROUP BY` หลายคอลัมน์ = จัดกลุ่มตาม "คู่ของค่า"

```
SELECT branch AS สาขา, pay_method AS วิธีชำระเงิน,
       COUNT(*) AS จำนวนรายการ,
       SUM(qty * unit_price - discount) AS ยอดขาย
FROM   sale_item
GROUP  BY branch, pay_method
ORDER  BY สาขา, ยอดขาย DESC;
```

สาขา	เงินสด	พร้อมเพย์	บัตรเครดิต	รวม
สาขาสีลม	13	**14**	2	29
สาขารังสิต	1	5	**11**	17
สาขาท่าพระจันทร์	5	**8**	1	14
รวม	19	27	14	**60**

ได้ 9 แถว (3 สาขา × 3 วิธีชำระ)

✅ **ไม่ใช่ "จัดกลุ่มตามสาขา แล้วค่อยจัดกลุ่มตามวิธีชำระ" — แต่คือ "จัดกลุ่มตามคู่ (สาขา, วิธีชำระ)"**

และตารางนี้บอกข้อมูลธุรกิจที่มีค่ามาก: สาขารังสิตแทบไม่มีใครจ่ายเงินสดเลย (1 จาก 17)

เพราะเป็นสาขาที่ขายทางเดลิเวอรีเป็นหลัก — ข้อมูลแบบนี้เห็นได้จาก `GROUP BY` สองคอลัมน์เท่านั้น

G10 — ⚠️ `NULL` รวมกันเป็นกลุ่มของมันเอง

```sql
SELECT member_code AS รหัสสมาชิก, COUNT(*) AS จำนวนรายการ,
       SUM(qty * unit_price - discount) AS ยอดสะสม
FROM   sale_item
GROUP  BY member_code
ORDER  BY ยอดสะสม DESC;
```

```
 รหัสสมาชิก | จำนวนรายการ | ยอดสะสม
------------+-------------+----------
 [null]     |          29 |  2930.00   ← กลุ่มที่ทำเงินให้ร้านมากที่สุด!
 M-007      |           4 |   535.00
 M-004      |           5 |   515.00
 M-006      |           2 |   350.00
 ...                                   → รวม 12 แถว
```

เปรียบเทียบพฤติกรรมของ `NULL` ให้ชัด

อยู่ใน	`NULL` ทำตัวอย่างไร
`WHERE`	ไม่ผ่านเงื่อนไขไหนเลย → **หายไปจากผลลัพธ์**
`GROUP BY`	ทุกแถวที่เป็น `NULL` ถือว่า "เหมือนกัน" → **รวมเป็นกลุ่มเดียว**

สองพฤติกรรมนี้ขัดกันเอง แต่ทั้งคู่ถูกต้องตามมาตรฐาน SQL — **ต้องจำทั้งสองอย่าง**

⚠️ สมาชิกมี 12 คน แต่ในผลลัพธ์มีสมาชิกจริงแค่ 11 เพราะ M-009 เพิ่งสมัครและยังไม่เคยซื้ออะไร

คนที่ไม่มีแถวในตาราง ย่อมไม่มีกลุ่มของตัวเอง — จะดึงออกมาด้วยต้องใช้ `JOIN` สัปดาห์หน้า

G11–G12 — `WHERE` กรองแถว · `HAVING` กรองกลุ่ม

```sql
SELECT category, COUNT(*) AS จำนวนรายการ, SUM(qty) AS แก้ว
FROM   sale_item
WHERE  branch = 'สาขาสีลม'          -- ① กรองแถวก่อนจัดกลุ่ม
GROUP  BY category;
-- ได้ 5 แถว รวมกันได้ 29 รายการ (ไม่ใช่ 60)
```

```sql
SELECT menu_name_th AS เมนู, COUNT(*) AS ขายกี่ครั้ง, SUM(qty) AS ขายกี่แก้ว
FROM   sale_item
GROUP  BY menu_name_th
HAVING COUNT(*) >= 4                -- ② กรองกลุ่มหลังจัดกลุ่ม
ORDER  BY ขายกี่ครั้ง DESC, เมนู ASC;
```

```
        เมนู         | ขายกี่ครั้ง | ขายกี่แก้ว
---------------------+-------------+------------
 ลาเต้ร้อน           |           6 |          8
 คาราเมลมัคคิอาโต้   |           5 |          9
 ลาเต้เย็น           |           5 |          9
 ครัวซองต์เนยสด      |           4 |          6
 ชาไทยเย็น           |           4 |          8
                      20 กลุ่ม → เหลือ 5 กลุ่ม
```

✅ `WHERE` กรอง "แถว" ก่อนจัดกลุ่ม → ใช้ฟังก์ชันรวมไม่ได้

✅ `HAVING` กรอง "กลุ่ม" หลังจัดกลุ่ม → ใช้ฟังก์ชันรวมได้ และมักต้องใช้

⚠️ G13 — สอง error ที่นักศึกษาทำผิดบ่อยที่สุด

❌ แบบที่ 1 — เอาฟังก์ชันรวมไปใส่ใน `WHERE`

```
SELECT category, COUNT(*) FROM sale_item WHERE COUNT(*) > 5 GROUP BY category;
```

```
ERROR:  aggregate functions are not allowed in WHERE
SQL state: 42803
```

ตอนที่ `WHERE` ทำงาน ยังไม่มีการจัดกลุ่มเกิดขึ้นเลย `COUNT(*)` ของกลุ่มจึงยังไม่มีอยู่

✅ ทางแก้: ย้ายไปไว้ใน `HAVING`

❌ แบบที่ 2 — ใส่คอลัมน์ที่ไม่ได้จัดกลุ่มและไม่ได้สรุป

```
SELECT menu_name_th, category, COUNT(*) FROM menu GROUP BY category;
```

```
ERROR:  column "menu.menu_name_th" must appear in the GROUP BY clause
        or be used in an aggregate function
SQL state: 42803
```

หมวด "กาแฟร้อน" มี 6 เมนู เมื่อยุบเหลือแถวเดียว **PostgreSQL ไม่รู้ว่าจะเอาชื่อเมนูไหนมาใส่ จึงปฏิเสธที่จะเดา**

✅ ทางแก้: ถ้าอยากได้ "ตัวแทนหนึ่งค่า" ต้องบอกให้ชัด เช่น `MIN(menu_name_th)` หรือ `MAX(menu_name_th)`

> **สอง error นี้เป็นรหัสเดียวกัน (42803) แต่คนละสาเหตุ — อ่านข้อความให้จบทุกครั้ง**

? ชวนคิดข้อที่ 4 — `WHERE` หรือ `HAVING`

เจ้าของร้านถามมา 5 ข้อ **แต่ละข้อควรเขียนเงื่อนไขไว้ที่ `WHERE` หรือ `HAVING` (หรือทั้งคู่)**

#	คำถามของเจ้าของร้าน
1	ขอดูยอดขายเฉพาะ**สาขาสีลม** แยกตามหมวด
2	ขอดูเฉพาะ**หมวดที่ขายได้เกิน 15 รายการ**
3	ขอดูยอดขาย**ของสัปดาห์นี้** แยกตามบาริสต้า
4	ขอดูเฉพาะ**เมนูที่ทำเงินได้เกิน 1,000 บาท**
5	ขอดูยอดขาย**กาแฟเย็น** เฉพาะ**สาขาที่ขายได้เกิน 15 แก้ว**

ใบ้: มีข้อหนึ่งที่ต้องใช้ทั้ง `WHERE` และ `HAVING` พร้อมกัน

✅ แนวคำตอบข้อที่ 4

#	คำตอบ	เงื่อนไขที่เขียน	เพราะอะไร
1	`WHERE`	`branch = 'สาขาสีลม'`	ตัดสินได้จากข้อมูลในแถวนั้นแถวเดียว
2	`HAVING`	`COUNT(*) > 15`	ต้องนับทั้งกลุ่มก่อนจึงจะรู้
3	`WHERE`	`sold_at >= TIMESTAMP '2026-09-14 00:00:00'`	แต่ละแถวมีวันเวลาของตัวเองอยู่แล้ว
4	`HAVING`	`SUM(qty * unit_price - discount) > 1000`	ต้องรวมทั้งกลุ่มก่อน
5	**ทั้งคู่**	`WHERE category = 'กาแฟเย็น'` + `HAVING SUM(qty) > 15`	กรองแถวก่อน แล้วค่อยกรองกลุ่ม

```sql
-- เฉลยข้อ 5 เขียนเต็ม
SELECT branch, COUNT(*) AS รายการ, SUM(qty) AS แก้ว,
       SUM(qty * unit_price - discount) AS ยอดขาย
FROM   sale_item
WHERE  category = 'กาแฟเย็น'
GROUP  BY branch
HAVING SUM(qty) > 15;
```

✅ **วิธีตัดสินใจที่ใช้ได้เสมอ ถามตัวเองแค่คำถามเดียว:**

"เงื่อนไขนี้มีฟังก์ชันรวม (`COUNT` `SUM` `AVG` `MIN` `MAX`) อยู่ข้างในไหม"

มี ⟶ `HAVING` · ไม่มี ⟶ `WHERE` · ถ้ามีเงื่อนไขทั้งสองแบบ ⟶ ใช้ทั้งคู่

G14 — ประกอบทุกอย่างของสัปดาห์นี้ไว้ในคำสั่งเดียว

โจทย์: *"ขอดูยอดขายกาแฟ แยกตามสาขาและประเภท เอาเฉพาะกลุ่มที่ขายได้ตั้งแต่ 10 แก้วขึ้นไป เรียงจากยอดมากไปน้อย เอา 5 อันดับแรก"*

```
SELECT branch   AS สาขา,
       category AS ประเภท,
       COUNT(*) AS จำนวนรายการ,
       SUM(qty) AS แก้วที่ขายได้,
       SUM(qty * unit_price - discount) AS ยอดขาย
FROM   sale_item                -- ① หยิบ 60 แถว
WHERE  category LIKE 'กาแฟ%'    -- ② เหลือ 36 แถว
GROUP  BY branch, category      -- ③ ยุบเหลือ 6 กลุ่ม
HAVING SUM(qty) >= 10           -- ④ เหลือ 3 กลุ่ม
ORDER  BY ยอดขาย DESC           -- ⑤ เรียง
LIMIT  5;                       -- ⑥ ตัด
```

```
       สาขา         | ประเภท    | จำนวนรายการ | แก้วที่ขายได้ | ยอดขาย
--------------------+-----------+-------------+---------------+---------
 สาขารังสิต         | กาแฟเย็น  |           7 |            11 |  905.00
 สาขาสีลม           | กาแฟเย็น  |           7 |            10 |  775.00
 สาขาสีลม           | กาแฟร้อน  |           9 |            10 |  680.00
```

ทิ้งไปที่ขั้นไหน	อะไรหายไป
② `WHERE`	ชา ขนม เครื่องดื่มอื่น รวม 24 แถว
④ `HAVING`	ท่าพระจันทร์/กาแฟร้อน (8 แก้ว) · ท่าพระจันทร์/กาแฟเย็น (8 แก้ว) · รังสิต/กาแฟร้อน (3 แก้ว)

ขอ 5 อันดับแต่ได้มา 3 เพราะมีแค่ 3 กลุ่มที่ผ่าน `HAVING` — `LIMIT` ไม่ได้บังคับว่าต้องได้ครบตามจำนวน

? ชวนคิดข้อที่ 5 — เขียนคำสั่งตอบโจทย์เจ้าของร้าน

เขียนคำสั่ง SQL เต็ม ๆ ตอบ 3 คำถามนี้ (ทำคู่กับเพื่อน 10 นาที)

1) "อยากรู้ว่าบาริสต้าคนไหนขายได้เงินเยอะที่สุด"

แสดงชื่อบาริสต้า จำนวนรายการ และยอดเงินรวม เรียงจากมากไปน้อย

2) "ขอรายชื่อเมนูลาเต้ทั้งหมด พร้อมราคาและกำไรต่อแก้ว เรียงจากกำไรมากไปน้อย"

ต้องได้ครบทั้ง 5 รายการที่มีคำว่า "ลาเต้" อยู่ในชื่อ

3) "สมาชิกระดับ gold กับ platinum คนไหนบ้างที่ยังไม่ได้ให้อีเมลไว้"

แสดงรหัส ชื่อ-สกุล ระดับ และแต้มสะสม เรียงตามแต้มจากมากไปน้อย

ใบ้: ข้อ 1 ใช้ `GROUP BY` · ข้อ 2 ใช้ `LIKE` + คอลัมน์คำนวณ + `ORDER BY`
ข้อ 3 ใช้ `IN` + `IS NULL`

✅ แนวคำตอบข้อที่ 5

```sql
-- 1) บาริสต้าคนไหนขายได้เงินเยอะที่สุด            → 4 แถว
SELECT barista AS บาริสต้า, COUNT(*) AS จำนวนรายการ, SUM(qty) AS แก้ว,
       SUM(qty * unit_price - discount) AS ยอดขาย
FROM   sale_item
GROUP  BY barista
ORDER  BY ยอดขาย DESC;
```

```sql
-- 2) เมนูลาเต้ทั้งหมด พร้อมกำไรต่อแก้ว             → 5 แถว
SELECT menu_name_th AS เมนู, category AS หมวด, price AS ราคา,
       price - cost AS กำไรต่อแก้ว
FROM   menu
WHERE  menu_name_th LIKE '%ลาเต้%'
ORDER  BY กำไรต่อแก้ว DESC, เมนู ASC;
-- ลาเต้มะพร้าวเย็น 55 · ชาเขียวมัทฉะลาเต้ 54 · ลาเต้น้ำผึ้งร้อน 51 ·
-- ลาเต้เย็น 48 · ลาเต้ร้อน 44
```

```sql
-- 3) สมาชิก gold/platinum ที่ยังไม่ได้ให้อีเมล    → 1 แถว
SELECT member_code, full_name, tier, points
FROM   customer
WHERE  tier IN ('gold', 'platinum')
  AND  email IS NULL
ORDER  BY points DESC;
-- M-006  ณัฐพล มั่นคง  gold  870
```

⚠️ **ข้อ 2 ต้องใช้ `'%ลาเต้%'` ไม่ใช่ `'ลาเต้%'`** ไม่งั้นจะได้แค่ 4 แถว ตกชาเขียวมัทฉะลาเต้ไป

ส่วนที่ 8

คาบปฏิบัติการและการบ้าน

แผนที่ไฟล์และเช็กลิสต์ก่อนออกจากห้อง

ลำดับ	ไฟล์	เนื้อหา	รันซ้ำได้ไหม
1	`01-setup-coffeedb.sql`	สร้างฐานข้อมูลและตาราง 3 ใบ	ได้
2	`02-insert.sql`	ข้อมูลตั้งต้น 92 แถว + ตัวอย่าง `INSERT` I1–I9b	ได้
3	`03-select-where.sql`	S1–S8 และ W1–W14	ได้ ไม่จำกัด
4	`04-like.sql`	L1–L13	ได้ ไม่จำกัด
5	`05-order-by.sql`	O1–O10b	ได้ ไม่จำกัด
6	`06-group-by.sql`	G1–G14	ได้ ไม่จำกัด
7	`07-แบบฝึกหัดท้ายคาบ.sql`	**แบบฝึกหัด 15 ข้อ พร้อมเฉลยท้ายไฟล์**	—

เช็กลิสต์ — ตรวจตัวเองก่อนออกจากห้อง

- [ ] `SELECT COUNT(*)` ทั้ง 3 ตารางได้ **20 / 12 / 60**
- [ ] อธิบายได้ว่าทำไม `WHERE email = NULL` จึงได้ 0 แถวโดยไม่มี error
- [ ] เขียน `LIKE` หาเมนูที่มีคำว่า "ลาเต้" ได้ **5 แถว** (ไม่ใช่ 4)
- [ ] รู้ว่า `LIKE '%latte%'` ได้ 0 แถว แต่ `ILIKE '%latte%'` ได้ 5 แถว เพราะอะไร
- [ ] เรียงด้วยคีย์รองเป็น และจัดตำแหน่ง `NULL` ด้วย `NULLS LAST` ได้
- [ ] บอกได้ว่าเงื่อนไขหนึ่ง ๆ ควรอยู่ `WHERE` หรือ `HAVING` โดยไม่ต้องเดา
- [ ] เขียนคำสั่ง G14 ที่ใช้ครบทุก clause ได้ด้วยตัวเอง
- [ ] ทำแบบฝึกหัดในไฟล์ `07` ได้อย่างน้อย **12 จาก 15 ข้อ**

สรุปสัปดาห์ที่ 8 — 6 คำสั่ง 6 บรรทัด

คำสั่ง	จำไว้ว่า	กับดักที่ต้องระวัง
`INSERT`	ระบุชื่อคอลัมน์เสมอ	อ่านชื่อ constraint ใน error ก่อนแก้
`SELECT`	`*` ใช้สำรวจ ไม่ใช้ในโปรแกรมจริง	`NULL` ในนิพจน์ `||` ทำให้ผลเป็น `NULL` ทั้งอัน
`WHERE`	กรอง **แถว** ก่อนทุกอย่าง	`AND` ทำงานก่อน `OR` · `= NULL` ได้ 0 แถวเงียบ ๆ
`LIKE`	`%` กี่ตัวก็ได้ · `_` หนึ่งตัวพอดี	แยกตัวพิมพ์ใหญ่-เล็ก · `NOT LIKE` ทิ้งแถว `NULL`
`ORDER BY`	ค่าซ้ำเมื่อไร ใส่คีย์รองเมื่อนั้น	`NULL` ตำแหน่งตั้งต้นต่างกันระหว่าง ASC/DESC
`GROUP BY`	ยุบหลายแถวเหลือหนึ่งแถวต่อกลุ่ม	`COUNT(*)` ≠ `COUNT(คอลัมน์)` · `NULL` เป็นกลุ่มเดียว

ลำดับการทำงานที่ต้องท่องให้ขึ้นใจ

```
FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT
```

ลำดับเดียวนี้อธิบายเรื่องแปลกทั้งหมดของวันนี้

ทำไม `ORDER BY` ใช้ชื่อแทนได้ แต่ `WHERE` ใช้ไม่ได้ · ทำไม `COUNT(*)` อยู่ใน `HAVING` ได้ แต่อยู่ใน `WHERE` ไม่ได้

เตรียมตัวสัปดาห์หน้า

สิ่งที่วันนี้ยังทำไม่ได้ — และจะแก้ด้วยอะไร

ปัญหาที่เจอวันนี้	สัปดาห์หน้าจะแก้ด้วย
`sale_item` ต้องเก็บชื่อเมนูซ้ำกับตาราง `menu`	`JOIN` — เชื่อมสองตารางเข้าด้วยกันตอนสืบค้น
M-009 ไม่เคยซื้อของ จึงไม่โผล่ในรายงาน `GROUP BY`	`LEFT JOIN` — ดึงคนที่ยอดเป็นศูนย์ออกมาได้
`AVG(kcal)` หารด้วย 18 แทนที่จะเป็น 20	`COALESCE` — แทนที่ `NULL` ด้วยค่าที่เรากำหนด
ต้องเขียน `menu_name_th || ' — ' || note` แล้วได้ `NULL`	`COALESCE` และฟังก์ชันข้อความอื่น ๆ

สิ่งที่ต้องเตรียมมา

1. **ฐานข้อมูล `coffeedb` ต้องยังอยู่ในเครื่อง** และมีข้อมูลครบ 20 / 12 / 60
   (ถ้าหาย ให้รันไฟล์ `01` และ `02` ใหม่ ใช้เวลาไม่ถึง 1 นาที)
2. **ทำแบบฝึกหัด 15 ข้อในไฟล์ `07` มาให้ครบ** — ข้อสอบกลางภาคจะออกในรูปแบบเดียวกัน
3. ทบทวนสไลด์ 3 หน้านี้เป็นพิเศษ: **กับดัก `5 + 4 ≠ 12`** · **`AND` ทำงานก่อน `OR`** · **`WHERE` เทียบกับ `HAVING`**

> **สอบกลางภาคสัปดาห์ที่ 9** — วัดว่า "ทำได้" ไม่ใช่แค่ "จำได้"
> ข้อสอบจะให้โจทย์ธุรกิจแล้วให้เขียนคำสั่ง SQL ไม่ใช่ให้ท่องนิยาม

คำถาม?

สัปดาห์ที่ 8 — การสืบค้นข้อมูลพื้นฐาน

ไฟล์ทั้งหมดอยู่ใน `เอกสารการสอน/โค้ด_สัปดาห์ที่8_CE396/`

สื่อสอนแบบโต้ตอบ: `สอนสด_สัปดาห์ที่8_CE396.html`

เปิดด้วยเบราว์เซอร์ได้เลย ไม่ต้องติดตั้งอะไร

อาจารย์สนายุ จินตนาวรรณกุล